Modul 3

Vertiefung SQL

Ausgewählte Operatoren und Funktionen

Josef Altmann





Der vorliegende Foliensatz basiert auf:
Kemper, A., Leis, V., Eickler, A.: *Datenbanksysteme*, De Gruyter Oldenbourg, 11. Auflage (2026)
Saake, G., Sattler, K.-U., Heuer, A.: *Datenbanken: Konzepte und Sprachen*, MITP, 6. Auflage (2018)

SQL-Standard Entwicklungshistorie

Jahr	Entwicklung
1975	*SEQUEL = Structured English Query Language*, der **Vorläufer** von *SQL*, wird für das Projekt **System R** von **IBM** entwickelt
1979	*SQL* gelangt mit *Oracle V2* erstmals durch *Relational Software Inc.* auf den **Markt**
1986	***SQL1*** wird von **ANSI** als Standard verabschiedet
1987	***SQL1*** wird von der Internationalen Organisation für Normung (**ISO**) als Standard verabschiedet und 1989 überarbeitet – Hinzufügung von **Integritätsbedingungen**
1992	Der Standard ***SQL2*** oder ***SQL-92*** wird von der **ISO** verabschiedet
1999	***SQL3*** oder ***SQL:1999*** wird verabschiedet. Features wie **Trigger**, rekursive Abfragen, reguläre Ausdrücke und OO-Erweiterungen werden hinzugefügt
2003	***SQL:2003*** fügt erste Änderungen für besseren **XML-Support** ein und führt die OVER-Klausel zur Unterstützung von Data Warehousing ein
2006	***SQL/XML:2006*** legt genauer fest, wie SQL in Zusammenhang mit XML verwendet werden kann
2008	***SQL:2008*** fügt **„INSTEAD OF"-Trigger** und das TRUNCATE- und FETCH FIRST-Statement hinzu
2011	***SQL:2011*** erweitert die OVER-Klausel und erweitert die analytischen Funktionen
2016	***SQL:2016*** ist die **aktuelle Revision** des Standards und führt LIST_AGG-Statement und JSON-Unterstützung ein.
2023	***SQL/PGQ*** unterstützt **Property Graph Queries**. **SQL/JSON**-Typ mit Funktionen.

Überblick

- Komplexe Gruppierung
  - **ROLLUP**- und **CUBE**-Operator
  - **GROUPING SETS**- und **GROUPING**-Operator
  - **PIVOT**- und **UNPIVOT**-Operator
- Analytische Anfragen
  - **OVER**- und **PARTITION BY**-Klausel
  - **OVER**- und **ORDER BY**-Klausel
  - Ranking-Funktionen
- Rekursive Anfragen
- Künstliche Schlüssel und Schlüsselgenerierung
  - Sequenzgeneratoren
  - Identitätsspalten
- Generierte Spalten
- Ausgewählte Funktionen und Anweisungen

© 2026

Gruppierung

Vorlesung

VorlNr	Titel	SWS	GelesenVon
5041	Ethik	4	2125
5049	Mäeutik	2	2125
4052	Logik	4	2125
5052	Wissenschaftstheorie	3	2126
5216	Bioethik	2	2126
5001	Grundzüge	4	2137
4630	Die 3 Kritiken	4	2137
...	...	...	...

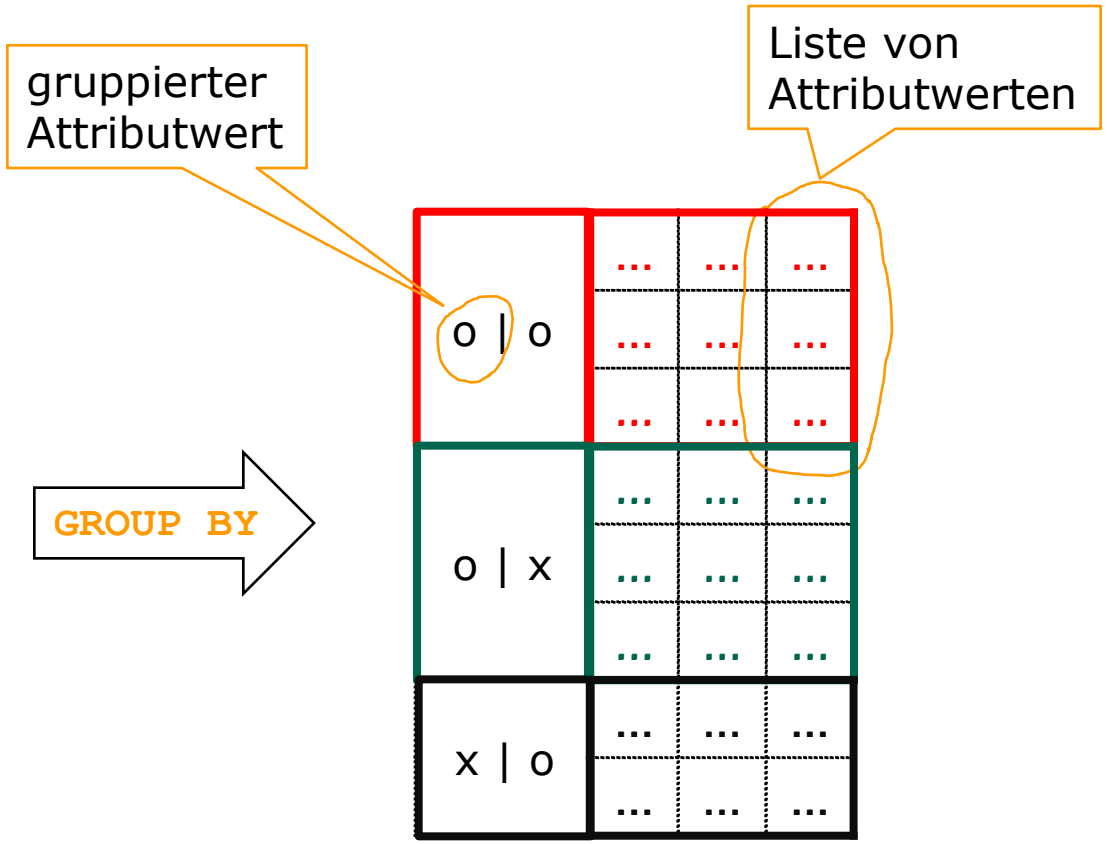
GROUP BY

GelesenVon	SUM(SWS)
2125	10
2126	5
2137	8
...	...

- Fragestellung
  - Wollen nicht die Summe aller Vorlesungsstunden absolut berechnen, sondern zu jedem/r Vortragenden die Summe der von ihm/ihr gehaltenen Stunden.
- Lösung
  - Bilde zu jedem/r Vortragenden eine Gruppe (mit der `GROUP BY`-Klausel) und summiere nur innerhalb dieser Gruppe.
- Allgemein
  - Alle Zeilen einer Tabelle, die auf den Attributen der `GROUP BY`-Klausel denselben Wert annehmen, werden zu einer Gruppe zusammengefasst und die Aggregatfunktionen werden bezüglich der Gruppe ausgewertet.
- Bedingungen
  - an die aggregierten Werte werden in der `HAVING`-Klausel angeführt.

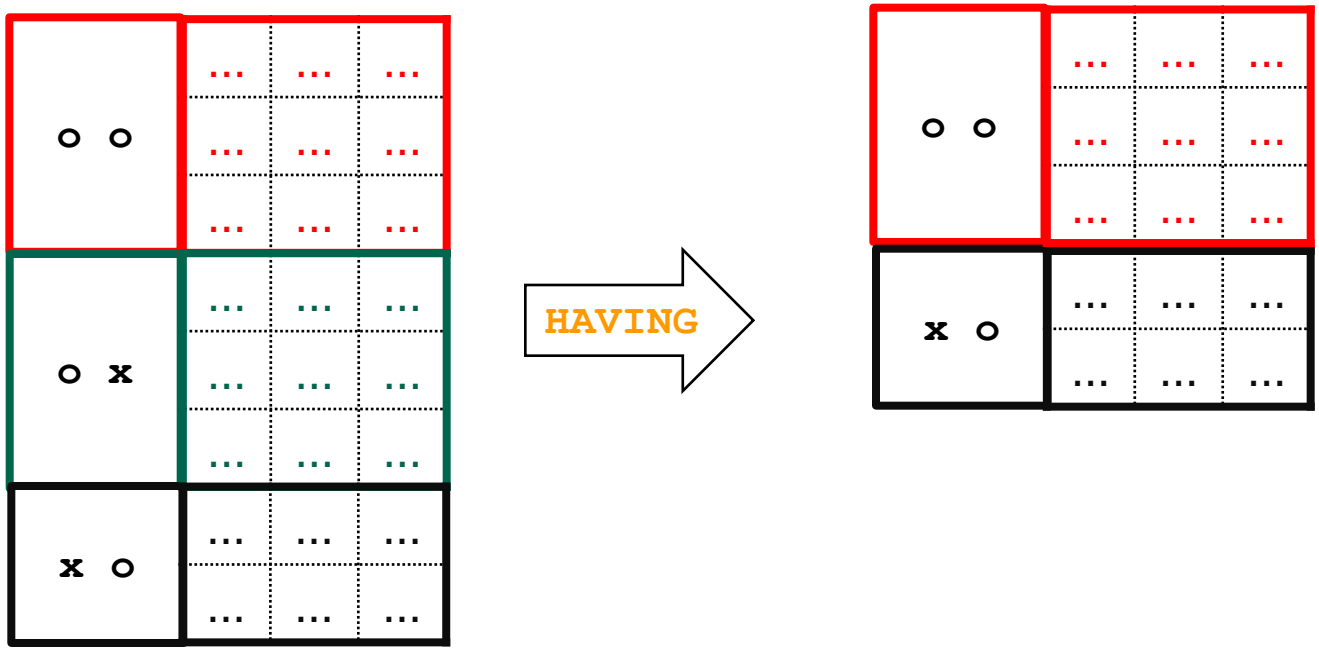
Gruppierung

o	o	…	…	…
o	o	…	…	…
o	x	…	…	…
o	o	…	…	…
x	o	…	…	…
o	x	…	…	…
o	x	…	…	…
x	o	…	…	…



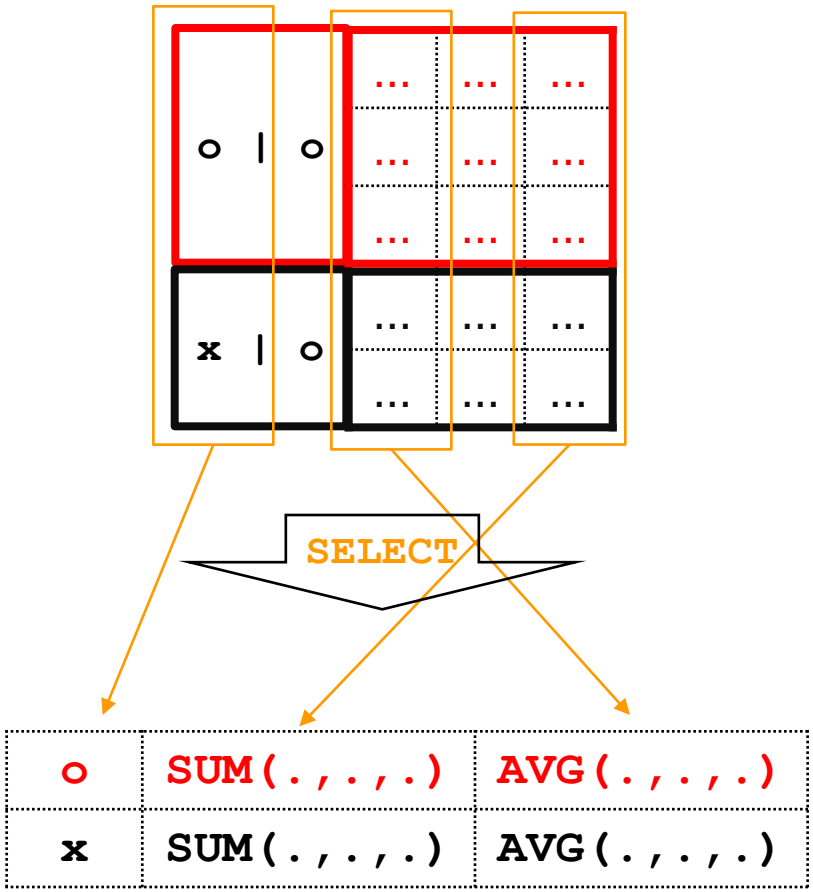
- Sobald **`GROUP BY`** verwendet wird, gibt es nur mehr
  - gruppierte Attributwerte (hier z.B. o und x) oder
  - Liste von Attributwerten (je z.B. eine Spalte mit […, …, …])

Gruppenauswahl



© 2026

Projektion und Aggregation



Gruppierung 1/6

GROUP BY- und HAVING-Klauseln

Notation:

```
SELECT ...
FROM ...
[ WHERE bedingungen ]          ──→  Auswahl Tabellenzeilen
[ GROUP BY attributliste ]     ──→  Gruppierungen von Tabellenzeilen
[ HAVING bedingungen ];        ──→  Auswahl Gruppierungen
```

- GROUP BY-Operator unterteilt die in der FROM-Klausel angegebene Tabelle in Gruppen, so dass alle Tupel innerhalb einer Gruppe den gleichen Wert für die GROUP BY-Attribute haben.
- Dabei werden alle Tupel, die die WHERE-Klausel nicht erfüllen, eliminiert.
- Danach werden innerhalb jeder Gruppe die Aggregatfunktionen berechnet.
- Die HAVING-Klausel ermöglicht es, Bedingungen an die durch GROUP BY gebildeten Gruppen zu formulieren.
- In der SELECT-Klausel dürfen nur zwei Spaltenarten vorkommen
  - die, die mit einer Aggregatfunktion versehen sind und
  - die anderen müssen in der GROUP BY-Klausel enthalten sein.

© 2026

Aggregate und Gruppierung 2/6

GROUP BY- und HAVING-Klauseln

REL

A	B	C	D
1	2	3	4
1	2	4	5
2	3	3	4
3	3	4	5
3	3	6	7
4	3	6	8



ERGEBNIS

A	SUM(D)
1	9

Anfrage:

```
SELECT A, SUM(D)
FROM REL
WHERE D <= 7
GROUP BY (A,  B)
HAVING A < 4 AND SUM(D) < 10 AND MAX(C) = 4;
```

Aggregate und Gruppierung 3/6

GROUP BY- und HAVING-Klauseln

- Schritt 1: **FROM** und **WHERE**

REL

A	B	C	D
1	2	3	4
1	2	4	5
2	3	3	4
3	3	4	5
3	3	6	7
4	3	6	8



WHERE-Klausel:
Bedingungen an einzelne Tupel bevor gruppiert wird

A	B	C	D
1	2	3	4
1	2	4	5
2	3	3	4
3	3	4	5
3	3	6	7

Anfrage:

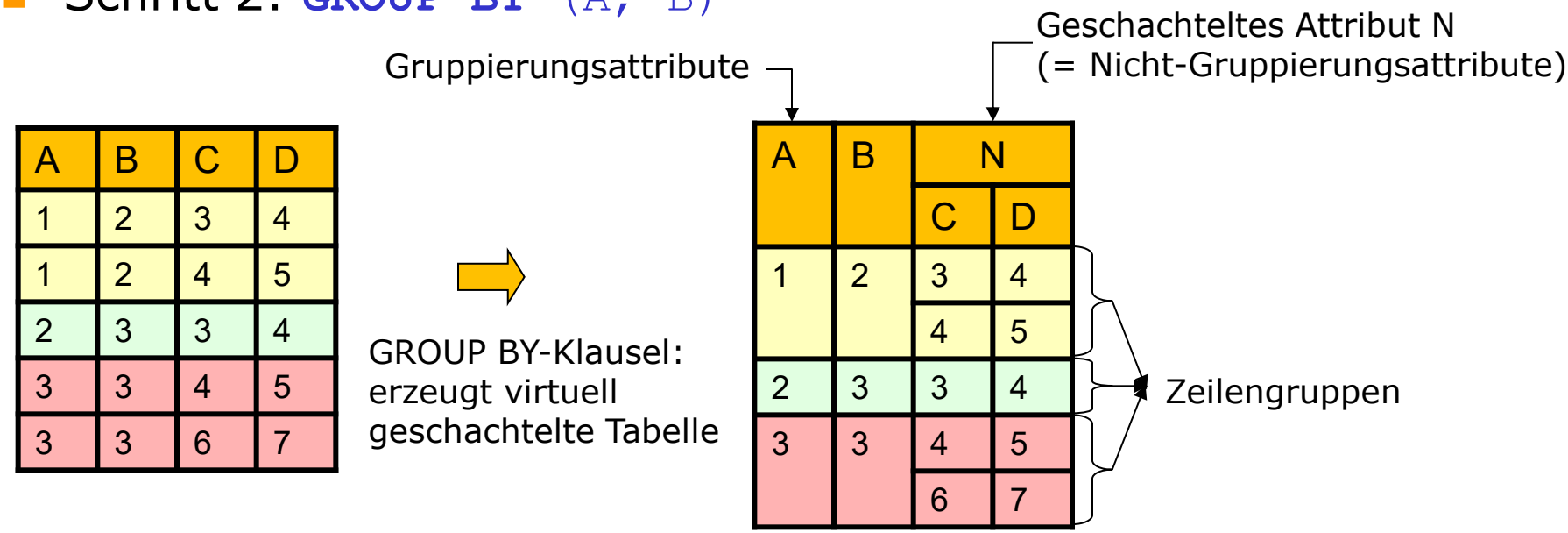
```
SELECT A, SUM(D)
FROM REL
WHERE D <= 7
GROUP BY (A,  B)
HAVING A < 4 AND SUM(D) < 10 AND MAX(C) = 4;
```

© 2026

Aggregate und Gruppierung 4/6

GROUP BY- und HAVING-Klauseln

- Schritt 2: **GROUP BY** (A, B)



Anfrage:

```
SELECT A, SUM(D)
FROM REL
WHERE D <= 7
GROUP BY (A,  B)
HAVING A < 4 AND SUM(D) < 10 AND MAX(C) = 4;
```

Aggregate und Gruppierung 5/6

GROUP BY- und HAVING-Klauseln

- Schritt 3: `HAVING A < 4 AND SUM(D) < 10 AND MAX(C) = 4`

A	B	N	
		C	D
1	2	3	4
		4	5
2	3	3	4
3	3	4	5
		6	7

Aggregation

A	B	N	
		SUM(D)	MAX(C)
1	2	9	4
2	3	4	3
3	3	12	6

HAVING-Klausel (=Selektion)

A	B	N	
		SUM(D)	MAX(C)
1	2	9	4

Anfrage:

```
SELECT A, SUM(D)
FROM REL
WHERE D <= 7
GROUP BY (A,  B)
HAVING A < 4 AND SUM(D) < 10 AND MAX(C) = 4;
```

In der HAVING-Klausel dürfen neben Aggregatfunktionen nur Attribute vorkommen, die explizit in der GROUP BY-Klausel aufgeführt wurden.

© 2026

Aggregate und Gruppierung 6/6

GROUP BY- und HAVING-Klauseln

- Schritt 4: **SELECT** A, **SUM**(D)

A	B	N	
		SUM(D)	MAX(C)
1	2	9	4



SELECT-Klausel
(=Projektion)

A	SUM(D)
1	9

Anfrage:

```
SELECT A, SUM(D)
FROM REL
WHERE D <= 7
GROUP BY (A,  B)
HAVING A < 4 AND SUM(D) < 10 AND MAX(C) = 4;
```

Aggregate und Gruppierung

- ACHTUNG
  - **GROUP BY**-Klausel erzeugt pro Gruppe **ein Ergebnistupel**
  - Alle in der **SELECT**-Klausel aufgeführten Attribute - außer den aggregierten - müssen auch in der **GROUP BY**-Klausel aufgeführt werden. So wird sichergestellt, dass die ausgegebenen Attribute sich nicht innerhalb der Gruppe ändern.
  - **GROUP BY**-Klausel betrachtet alle Nullwerte als wäre sie identisch. Nullwerte in aggregierten Attributen werden als Gruppe zusammengefasst.

```
SELECT gelesenVon, Name, SUM(SWS)
...
GROUP BY (gelesenVon, Name);
```

© 2026

Gruppierung

- **`GROUP BY`-Klausel** bewirkt Teilmengenbildung (Gruppierung) der Ergebnis-Relation entsprechend Gruppierungsattribut(en).
- Man kann die **`WHERE`-Klausel nicht** verwenden, um **Gruppen einzuschränken**.
- Man verwendet die **`HAVING`-Klausel**, um **Gruppen einzuschränken**. `HAVING` kann nur in Verbindung mit `GROUP BY` auftreten.
- Aggregatfunktionen werden auf Teilmengen angewandt.
- Als **Aggregatfunktionen** dürfen in der **`HAVING`-Klausel** `MIN`, `MAX`, `COUNT`, `SUM` und `AVG` verwendet werden.
- Man kann **keine Aggregatfunktionen** in der `WHERE`-Klausel verwenden
  - außer in der `WHERE`-Klausel ist wieder ein SFW-Block mit Aggregatfunktion enthalten
- Verbund wird vor der Gruppierung ausgeführt.

© 2026

Gruppierung

Welche der folgenden Anfragen sind korrekt?



Anfrage 1:

```
SELECT Filiale, SUM(Guthaben)
FROM Konto
GROUP BY (Filiale)
HAVING Guthaben < 700;
```



Anfrage 2:

```
SELECT Filiale, SUM(Guthaben)
FROM Konto
WHERE Filiale = 'Wels' OR
Filiale = 'Hagenberg'
GROUP BY (Filiale);
```



Anfrage 3:

```
SELECT Filiale, SUM(Guthaben)
FROM Konto
GROUP BY (Filiale)
HAVING SUM(Guthaben) < 1000;
```



Anfrage 4:

```
SELECT *
FROM Konto
GROUP BY (Filiale)
HAVING SUM(Guthaben) < 1000;
```



Konto

KontoNr	Filiale	Guthaben
101	Wels	500
215	Linz	700
102	Linz	400
305	Linz	350
222	Linz	700
201	Hagenberg	900
217	Hagenberg	750

© 2026

Gruppierung

Welches Ergebnis liefert die folgende Anfrage?



Anfrage 1:

```
SELECT Filiale, SumGuthaben
FROM ( SELECT Filiale, SUM(Guthaben) AS SumGuthaben
       FROM Konto
       GROUP BY (Filiale))
WHERE SumGuthaben > 1200;
```

Filiale	Guthaben
Linz	2150
Hagenberg	1650

Konto

KontoNr	Filiale	Guthaben
101	Wels	500
215	Linz	700
102	Linz	400
305	Linz	350
222	Linz	700
201	Hagenberg	900
217	Hagenberg	750

Anfrage 2 äquivalent zu 1:

```
SELECT Filiale, SUM(Guthaben) AS SumGuthaben
FROM Konto
GROUP BY (Filiale)
HAVING SUM(Guthaben) > 1200;
```

© 2026

Aggregatfunktionen

… und zum Abschluss noch etwas Statistik!



Beispiel:

Ermitteln Sie die das arithmetische Mittel, den Median, die Standard-abweichung sowie die Varianz für die Körpergröße der Studierenden?

```
SELECT  AVG(Groesse) AS "Arithmetisches Mittel",
        MEDIAN(Groesse) AS "Median",
        ROUND(STDDEV(Groesse),2) AS "Standardabweichung",
        ROUND(VARIANCE(Groesse),2) AS "Varianz/Streuung"
FROM Student;
```

Student

MatrNr	Name	Semester	Groesse
24002	Xenokrates	18	175
25403	Jonas	12	195
26120	Fichte	10	162
26830	Aristoxenos	8	174
27550	Schopenhauer	6	189
28106	Carnap	3	178
29120	Theophrastos	2	165
29555	Feuerbach	2	170

Ergebnis

Arithmetisches Mittel	Median	Standard-abweichung	Varianz/ Streuung
176	174,5	11,29	127,43

Komplexe Gruppierung
Erweiterung der `GROUP BY`-Klausel

- **SQL:1999** führt `ROLLUP`- und `CUBE`-Operator zur Unterstützung von komplexen Gruppierungs- und Aggregationsmöglichkeiten ein (z.B. für umfangreiche Berichterstellung, Data Warehouse Anwendungen) um
  - die Anfrageformulierung zu vereinfachen und
  - eine effiziente Auswertung zu gewährleisten.

- Bei der Gruppierung mit `ROLLUP` wird eine Ergebnismenge erstellt, welche die
  - **gruppierten Zeilen** und die **Zwischensummenwerte** enthält.
- Bei der Gruppierung mit `CUBE` wird eine Ergebnismenge erstellt, welche die
  - **Zeilen aus der `ROLLUP`-Operation und Kreuztabellenzeilen** enthält.

Komplexe Gruppierung
ROLLUP-Operator

- **ROLLUP** ist eine Erweiterung der **GROUP BY**-Klausel.
- Mit dem **ROLLUP-Operator** können **kumulative Aggregate**, z.B. Zwischensummen, erstellt werden.

```
SELECT A, B, SUM(D) AS Summe
FROM REL
WHERE D <= 7
GROUP BY ROLLUP (A, B);
```

Ohne ROLLUP-Operator: n +1 Unteranfragen

```
SELECT A, B, SUM(D) AS Summe
  FROM REL
 WHERE D <= 7
GROUP BY (A,B)
UNION ALL
SELECT A, NULL AS B, SUM(D)
  FROM REL
 WHERE D <= 7
GROUP BY (A)
UNION ALL
SELECT NULL AS A, NULL AS B, SUM(D)
  FROM REL
 WHERE D <= 7
ORDER BY A,B;
```

A	B	C	D
1	2	3	4
1	2	4	5
1	3	4	5
2	3	3	4
3	3	4	5
3	4	6	7
4	3	6	8

ROLLUP

A	B	SUMME
1	2	9
1	3	5
1	null	14
2	3	4
2	null	4
3	3	5
3	4	7
3	null	12
null	null	30

Superaggregate für A (= 1,2,3)

Gesamtsumme

Komplexe Gruppierung
ohne ROLLUP-Operator



Beispiel:

Ermitteln Sie das minimale und maximale Gehalt sowie die Gehaltsumme und die Anzahl der Mitarbeiter gruppiert nach Abteilung und Beruf?

```
SELECT  department_id, job_id,
        MIN(salary),MAX(salary),SUM(salary),COUNT(salary)
FROM employees
GROUP BY(department_id, job_id)
ORDER BY department_id, job_id;
```

	EMPLOYEE_ID	FIRST_NAME	LAST_NAME	EMAIL	PHONE_NUMBER	HIRE_DATE	JOB_ID	SALARY	COMMISSION_PCT	MANAGER_ID	DEPARTMENT_ID
1	100	Steven	King	SKING	515.123.4567	17-JUN-87	AD_PRES	24000	NULL	NULL	90
2	101	Neena	Kochhar	NKOCHHAR	515.123.4568	21-SEP-89	AD_VP	17000	NULL	100	90
3	102	Lex	De Haan	LDEHAAN	515.123.4569	13-JAN-93	AD_VP	17000	NULL	100	90
18	205	Shelley	Higgins	SHIGGINS	515.123.8080	07-JUN-94	AC_MGR	12000	NULL	101	110
19	206	William	Gietz	WGIETZ	515.123.8181	07-JUN-94	AC_ACCOUNT	8300	NULL	205	110

ohne Zwischensummenwerte

	DEPARTMENT_ID	JOB_ID	MIN(SALARY)	MAX(SALARY)	SUM(SALARY)	COUNT(JOB_ID)
1	10	AD_ASST	4400	4400	4400	1
2	20	MK_MAN	13000	13000	13000	1
3	20	MK_REP	6000	6000	6000	1
4	50	ST_CLERK	2500	3500	11700	4
5	50	ST_MAN	5800	5800	5800	1
6	60	IT_PROG	4200	9000	19200	3
7	80	SA_MAN	10500	10500	10500	1
8	80	SA_REP	7000	11000	26600	3
9	90	AD_PRES	24000	24000	24000	1
10	90	AD_VP	17000	17000	34000	2
11	110	AC_ACCOUNT	8300	8300	8300	1
12	110	AC_MGR	12000	12000	12000	1

© 2026

Komplexe Gruppierung
mit ROLLUP-Operator



Beispiel:

Ermitteln Sie das minimale und maximale Gehalt sowie die Gehaltsumme und die Anzahl der Mitarbeiter gruppiert nach Abteilung und Beruf? Zusätzlich sollen die Aggregationen pro Abteilung ohne Berücksichtigung des Berufs sowie die Gehaltssumme über das gesamte Unternehmen ermittelt werden.

```
SELECT  department_id, job_id,
        MIN(salary),MAX(salary),SUM(salary),COUNT(salary)
FROM employees
GROUP BY ROLLUP(department_id, job_id)
ORDER BY department_id, job_id;
```

mit ROLLUP

	DEPARTMENT_ID	JOB_ID	MIN(SALARY)	MAX(SALARY)	SUM(SALARY)	COUNT(SALARY)
1	10	AD_ASST	4400	4400	4400	1
2	10	NULL	4400	4400	4400	1
3	20	MK_MAN	13000	13000	13000	1
4	20	MK_REP	6000	6000	6000	1
5	20	NULL	6000	13000	19000	2
6	50	ST_CLERK	2500	3500	11700	4
7	50	ST_MAN	5800	5800	5800	1
8	50	NULL	2500	5800	17500	5
9	60	IT_PROG	4200	9000	19200	3
10	60	NULL	4200	9000	19200	3
11	80	SA_MAN	10500	10500	10500	1
12	80	SA_REP	7000	11000	26600	3
13	80	NULL	7000	11000	37100	4
14	90	AD_PRES	24000	24000	24000	1
15	90	AD_VP	17000	17000	34000	2
16	90	NULL	17000	24000	58000	3
17	110	AC_ACCOUNT	8300	8300	8300	1
18	110	AC_MGR	12000	12000	12000	1
19	110	NULL	8300	12000	20300	2
20	NULL	NULL	2500	24000	175500	20

Es ist, als hätte man die Anweisung der Klausel **GROUP BY** in drei Durchläufen folgendermaßen geändert:

- `GROUP BY (department_id, job_id)`
- `GROUP BY (department_id)`
- `GROUP BY ()`

© 2026

Komplexe Gruppierung
CUBE-Operator

- **CUBE** ist eine Erweiterung der **GROUP BY**-Klausel
- Mit dem **CUBE**-Operator können mit einer einzelnen **SELECT**-Anweisung **Kreuztabellenwerte** erstellt werden.

```
SELECT A, B, SUM(D) AS Summe
FROM REL
WHERE D <= 7
GROUP BY CUBE (A, B);
```

Ohne CUBE-Operator: 2^2 = 4 Unteranfragen

```
SELECT A, B, SUM(D) AS Summe
  FROM REL
 WHERE D <= 7
GROUP BY (A,B)
UNION ALL
SELECT A, NULL AS B, SUM(D)
  FROM REL
 WHERE D <= 7
GROUP BY (A)
UNION ALL
SELECT NULL AS A, B, SUM(D)
  FROM REL
 WHERE D <= 7
GROUP BY (B)
UNION ALL
SELECT NULL AS A, NULL AS B, SUM(D)
  FROM REL
 WHERE D <= 7 ORDER BY A,B;
```

A	B	C	D
1	2	3	4
1	2	4	5
1	3	4	5
2	3	3	4
3	3	4	5
3	4	6	7
4	3	6	8



A	B	SUMME	
null	null	30	Gesamtsumme
null	2	9	nach B zusammen-
null	3	14	gefasst
null	4	7	
1	null	14	nach A zusammen-
1	2	9	gefasst
1	3	5	nach A und B zusammengefasst
2	null	4	
2	3	4	
3	null	12	Zwischensummen-
3	3	5	kombinationen
3	4	7	

Komplexe Gruppierung
mit CUBE-Operator



Beispiel:

Ermitteln Sie das minimale und maximale Gehalt sowie die Gehaltsumme und die Anzahl der Mitarbeiter gruppiert nach Abteilung und Beruf? Zusätzlich sollen die Aggregationen pro Abteilung ohne Berücksichtigung des Berufs sowie die Gehaltssumme über das gesamte Unternehmen ermittelt werden.
Weiters sollen auch die Aggregationen je Beruf ohne Berücksichtigung der Abteilung ermittelt werden.

```
SELECT  department_id,job_id,
        MIN(salary),MAX(salary),
        SUM(salary),COUNT(salary)
FROM employees
GROUP BY CUBE (department_id, job_id)
ORDER BY department_id, job_id;
```

CUBE

	DEPARTMENT_ID	JOB_ID	MIN(SALARY)	MAX(SALARY)	SUM(SALARY)	COUNT(SALARY)
1	10	AD_ASST	4400	4400	4400	1
2	10	NULL	4400	4400	4400	1
3	20	MK_MAN	13000	13000	13000	1
4	20	MK_REP	6000	6000	6000	1
5	20	NULL	6000	13000	19000	2
6	50	ST_CLERK	2500	3500	11700	4
7	50	ST_MAN	5800	5800	5800	1
8	50	NULL	2500	5800	17500	5
9	60	IT_PROG	4200	9000	19200	3
10	60	NULL	4200	9000	19200	3
11	80	SA_MAN	10500	10500	10500	1
12	80	SA_REP	7000	11000	26600	3
13	80	NULL	7000	11000	37100	4
14	90	AD_PRES	24000	24000	24000	1
15	90	AD_VP	17000	17000	34000	2
16	90	NULL	17000	24000	58000	3
17	110	AC_ACCOUNT	8300	8300	8300	1
18	110	AC_MGR	12000	12000	12000	1
19	110	NULL	8300	12000	20300	2
20	NULL	AC_ACCOUNT	8300	8300	8300	1
21	NULL	AC_MGR	12000	12000	12000	1
22	NULL	AD_ASST	4400	4400	4400	1
23	NULL	AD_PRES	24000	24000	24000	1
24	NULL	AD_VP	17000	17000	34000	2
25	NULL	IT_PROG	4200	9000	19200	3
26	NULL	MK_MAN	13000	13000	13000	1
27	NULL	MK_REP	6000	6000	6000	1
28	NULL	SA_MAN	10500	10500	10500	1
29	NULL	SA_REP	7000	11000	26600	3
30	NULL	ST_CLERK	2500	3500	11700	4
31	NULL	ST_MAN	5800	5800	5800	1
32	NULL	NULL	2500	24000	175500	20

Zusätzlich vom CUBE–Operator
Erzeugte Kreuztabellenwerte

© 2026

Komplexe Gruppierung

ROLLUP- versus CUBE-Operator

- **ROLLUP**-Operator generiert weniger Kombinationen, d.h. für `n` Attribute
  - bei `ROLLUP` ⇨ `n + 1` Kombinationen (Unteranfragen)
  - bei `CUBE` ⇨ 2^n Kombinationen (Unteranfragen)

  Gruppierungskombinationen

- Reihenfolgen der Attribute bei **ROLLUP**-Operator wichtig
  - `ROLLUP(`A_1`, `A_2`) <> ROLLUP(`A_2`, `A_1`)`

- **CUBE**- und **ROLLUP**-Operator generieren automatisch die Gruppierungskombinationen
  - `GROUPING SETS` unterstützen explizite Auswahl der Kombinationen

© 2026

Komplexe Gruppierung
GROUPING SETS

- Explizite Angabe der Gruppierungskombinationen

A	B	C	D
1	2	3	4
1	2	4	5
1	3	4	5
2	3	3	4
3	3	4	5
3	4	6	7
4	3	6	8

äquivalent zu

```
SELECT A, B, SUM(D) AS Summe
FROM REL
WHERE D <= 7
GROUP BY ROLLUP (A, B);
```

```
SELECT A, B, SUM(D) AS Summe
FROM REL
WHERE D <= 7
GROUP BY GROUPING SETS ((A, B), (A), ());
```

A	B	SUMME
1	2	9
1	3	5
1	null	14
2	3	4
2	null	4
3	3	5
3	4	7
3	null	12
null	null	30

```
SELECT A, B, SUM(D) AS Summe
FROM REL
WHERE D <= 7
GROUP BY GROUPING SETS ((A, B),());
```

A	B	SUMME
1	2	9
1	3	5
2	3	4
3	3	5
3	4	7
null	null	30

Komplexe Gruppierung
GROUPING-Operator

- Zusammen mit **CUBE**- oder **ROLLUP**-Operator verwendet
- Zur Ermittlung der Gruppen, welche die Zwischensumme in einer Zeile bilden
- Unterscheidet gespeicherte **NULL**-Werte von **NULL**-Werten, die mit dem **ROLLUP**- oder **CUBE**-Operator erstellt wurden

GROUPING:

```
SELECT A, B, SUM(D) AS Summe,
  GROUPING(A) GRP_A,
  GROUPING(B) GRP_B
FROM REL
WHERE D <= 7
GROUP BY CUBE(A, B);
```



A	B	SUMME	GRP_A	GRP_B
null	null	30	1	1
null	2	9	1	0
null	3	14	1	0
null	4	7	1	0
1	null	14	0	1
1	2	9	0	0
1	3	5	0	0
2	null	4	0	1
2	3	4	0	0
3	null	12	0	1
3	3	5	0	0
3	4	7	0	0

0 ... Spalte wurde zum Berechnen des Aggregatwertes verwendet. ODER Der NULL-Wert in der Spalte ist ein gespeicherter NULL-Wert.

1 ... Spalte wurde nicht zum Berechnen des Aggregatwertes verwendet. ODER Der NULL-Wert in der Spalte wurde von ROLLUP oder CUBE als Gruppierungsergebnis erstellt.

Komplexe Gruppierung

GROUPING-Operator mit CASE-Anweisung

GROUPING mit CASE:

```
SELECT
  CASE
    WHEN GROUPING(A) = 1 THEN 'Group A'
    ELSE to_char(A)
  END AS A,
  CASE
    WHEN GROUPING(B) = 1 THEN 'Group B'
    ELSE to_char(B)
  END AS B,
  SUM(D) AS Summe
FROM REL
WHERE D <= 7
GROUP BY CUBE(A, B);
```

A	B	C	D
1	2	3	4
1	2	4	5
1	3	4	5
1	null	1	2
2	3	3	4
3	3	4	5
3	4	6	7
4	3	6	8



A	B	SUMME
Group A	null	2
Group A	Group B	32
Group A	2	9
Group A	3	14
Group A	4	7
1	null	2
1	Group B	16
1	2	9
1	3	5
2	Group B	4
2	3	4
3	Group B	12
3	3	5
3	4	7

© 2026

Komplexe Gruppierung
mit GROUPING-Operator



Beispiel:

Ermitteln Sie die Gehaltssumme je Beruf mit und ohne Berücksichtigung der Abteilung sowie die Gehaltssumme je Abteilung und für das gesamte Unternehmen.

```
SELECT CASE GROUPING(d.department_name)
            WHEN 1 THEN 'Alle Abteilungen'
            ELSE d.department_name END AS "Abteilung",
       CASE GROUPING(e.job_id)
            WHEN 1 THEN 'Alle Berufe'
            ELSE e.job_id END AS "Beruf",
       GROUPING(d.department_name) AS "dept_group",
       GROUPING(e.job_id) AS "job_group",
       SUM(e.salary) AS "summe"
FROM   departments d JOIN employees e
       ON d.department_id=e.department_id
GROUP BY CUBE(d.department_name, e.job_id);
```

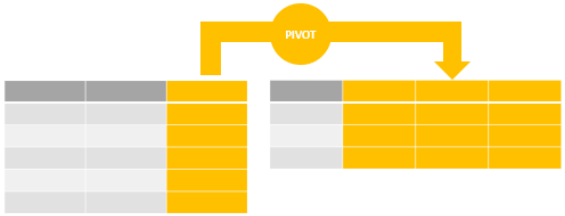
CUBE mit GROUPING

	Abteilung	Beruf	dept_group	job_group	summe
1	Alle Abteilungen	Alle Berufe	1	1	168500
2	Alle Abteilungen	AD VP	1	0	34000
3	Alle Abteilungen	AC MGR	1	0	12000
4	Alle Abteilungen	MK MAN	1	0	13000
5	Alle Abteilungen	MK REP	1	0	6000
6	Alle Abteilungen	SA MAN	1	0	10500
7	Alle Abteilungen	SA REP	1	0	19600
8	Alle Abteilungen	ST MAN	1	0	5800
9	Alle Abteilungen	AD ASST	1	0	4400
10	Alle Abteilungen	AD PRES	1	0	24000
11	Alle Abteilungen	IT PROG	1	0	19200
12	Alle Abteilungen	ST CLERK	1	0	11700
13	Alle Abteilungen	AC ACCOUNT	1	0	8300
14	IT	Alle Berufe	0	1	19200
15	IT	IT PROG	0	0	19200
16	Sales	Alle Berufe	0	1	30100
17	Sales	SA MAN	0	0	10500
18	Sales	SA REP	0	0	19600
19	Shipping	Alle Berufe	0	1	17500
20	Shipping	ST MAN	0	0	5800
21	Shipping	ST CLERK	0	0	11700
22	Executive	Alle Berufe	0	1	58000
23	Executive	AD VP	0	0	34000
24	Executive	AD PRES	0	0	24000
25	Marketing	Alle Berufe	0	1	19000
26	Marketing	MK MAN	0	0	13000
27	Marketing	MK REP	0	0	6000
28	Accounting	Alle Berufe	0	1	20300
29	Accounting	AC MGR	0	0	12000
30	Accounting	AC ACCOUNT	0	0	8300
31	Administration	Alle Berufe	0	1	4400
32	Administration	AD ASST	0	0	4400

© 2026

Komplexe Gruppierung

PIVOT-Operator – Zeilen als Spalten



- **Vertauscht Zeilen** und **Spalten** und ermöglicht eine einfache Datenanalyse aus unterschiedlichen Perspektiven
- Führt **implizit** eine **Gruppierung** durch
- **Gute Laufzeit** aber nicht standardisiert
- Beispiel: Gruppiere nach Feld „Kunde"
  - Alle Datensätze, die in der Spalte „Kunde" den gleichen Inhalt aufweisen,
  - werden zu je einem Datensatz zusammengefasst.

Verkauf

ID	Kunde	Produkt	Menge
1	Kunde A	Kalender	100
2	Kunde B	Block	40
3	Kunde C	Heft	70
4	Kunde D	Block	100
5	Kunde A	Block	200
6	Kunde D	Heft	30
7	Kunde D	Heft	10

Drehen/Umstellen von Spalten und Zeilen + Gruppieren + Aggregieren

Kunde	Kalender	Block	Heft
Kunde A	100	200	
Kunde B		40	
Kunde C			70
Kunde D		100	40

Komplexe Gruppierung

PIVOT-Operator – Zeilen als Spalten

Verkauf

ID	Kunde	Produkt	Menge
1	Kunde A	Kalender	100
2	Kunde B	Block	40
3	Kunde C	Heft	70
4	Kunde D	Block	100
5	Kunde A	Block	200
6	Kunde D	Heft	30
7	Kunde D	Heft	10

- **PIVOT**-Klausel erfordert die Angabe der
  - Art der Aggregation(en)
  - **FOR**-Klausel: Inhalte welcher Spalte zu Zeilen werden sollen
  - **IN**-Klausel: Welche Werte innerhalb der Spalte verwendet werden sollen

```
SELECT * FROM (
    SELECT kunde, produkt, menge FROM VERKAUF )
  PIVOT
   ( SUM(menge) AS SUM,
     COUNT(menge) AS ANZ
     FOR produkt
     IN (  'Kalender' AS KAL,
           'Block' AS BLOCK,
           'Heft' AS HEFT) )
ORDER BY kunde;
```

Inner View enthält jene Spalten, die für die Pivotierung benötigt werden.

Spalte(n) für die Aggregation(en)

Spalte für die Pivotierung

Werte der Pivot-Spalte, die in Spalten umgewandelt werden.

„Pivotierte Tabelle"

KUNDE	KAL_SUM	KAL_ANZ	BLOCK_SUM	BLOCK_ANZ	HEFT_SUM	HEFT_ANZ
Kunde A	100	1	200	1	null	0
Kunde B	null	0	40	1	null	0
Kunde C	null	0	null	0	70	1
Kunde D	null	0	100	1	40	2

© 2026

Komplexe Gruppierung

PIVOT-Operator – Zeilen als Spalten



Beispiel:

Erstellen Sie einen Bericht, welcher die Abteilungen 50, 60, 80 und 90 als Spalten definiert und die Berufe und Gehaltssumme als Zeilen darstellt?

	EMPLOYEE_ID	FIRST_NAME	LAST_NAME	EMAIL	PHONE_NUMBER	HIRE_DATE	JOB_ID	SALARY	COMMISSION_PCT	MANAGER_ID	DEPARTMENT_ID
1	100	Steven	King	SKING	515.123.4567	17-JUN-87	AD_PRES	24000	NULL	NULL	90
2	101	Neena	Kochhar	NKOCHHAR	515.123.4568	21-SEP-89	AD_VP	17000	NULL	100	90
18	205	Shelley	Higgins	SHIGGINS	515.123.8080	07-JUN-94	AC_MGR	12000	NULL	101	110
19	206	William	Gietz	WGIETZ	515.123.8181	07-JUN-94	AC_ACCOUNT	8300	NULL	205	110

```
SELECT * FROM (
       SELECT job_id, department_id, salary
       FROM employees)
    PIVOT
    (SUM(salary)
    FOR department_id
    IN ( 50 AS dept_50,
         60 AS dept_60,
         80 AS dept_80,
         90 AS dept_90)
);
```

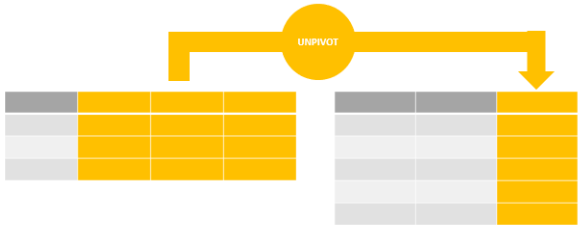
PIVOT

	JOB_ID	DEPT_50	DEPT_60	DEPT_80	DEPT_90
1	IT_PROG	NULL	19200	NULL	NULL
2	AC_MGR	NULL	NULL	NULL	NULL
3	AC_ACCOUNT	NULL	NULL	NULL	NULL
4	ST_MAN	5800	NULL	NULL	NULL
5	AD_ASST	NULL	NULL	NULL	NULL
6	AD_VP	NULL	NULL	NULL	34000
7	SA_MAN	NULL	NULL	10500	NULL
8	MK_MAN	NULL	NULL	NULL	NULL
9	AD_PRES	NULL	NULL	NULL	24000
10	SA_REP	NULL	NULL	26600	NULL
11	MK_REP	NULL	NULL	NULL	NULL
12	ST_CLERK	11700	NULL	NULL	NULL

© 2026

Komplexe Gruppierung

UNPIVOT-Operator – Spalten als Zeilen




```
CREATE TABLE REPORT AS
SELECT * FROM
    (SELECT kunde, produkt, menge FROM VERKAUF)
  PIVOT
  (
   SUM(menge)
   FOR produkt
   IN ('Kalender' AS kalender,
       'Block' AS block,
       'Heft' AS heft)
  )
ORDER BY kunde;
```

- **UNPIVOT**-Klausel erfordert die Angabe der
  - Spaltenüberschrift für die Werte
  - Spaltenüberschrift für die Spalten, die zu Zeilen werden sollen
  - Liste der Spalten, die einbezogen werden sollen

REPORT

KUNDE	KALENDER	BLOCK	HEFT
Kunde A	100	200	null
Kunde B	null	40	null
Kunde C	null	null	70
Kunde D	null	100	40

```
SELECT * FROM
    (SELECT kunde, kalender, block, heft FROM REPORT)
  UNPIVOT
  ( menge
    FOR produkt
    IN (kalender, block, heft) );
```

```
SELECT * FROM REPORT
  UNPIVOT INCLUDE NULLS
  ( menge
    FOR produkt
    IN (kalender, block, heft)
);
```

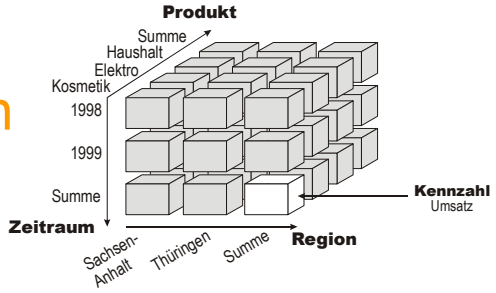
KUNDE	PRODUKT	MENGE
Kunde A	KALENDER	100
Kunde A	BLOCK	200
Kunde A	HEFT	null
Kunde B	KALENDER	null
Kunde B	BLOCK	40
Kunde B	HEFT	null
…	…	…

KUNDE	PRODUKT	MENGE
Kunde A	KALENDER	100
Kunde A	BLOCK	200
Kunde B	BLOCK	40
Kunde C	HEFT	70
Kunde D	BLOCK	100
Kunde D	HEFT	40

© 2026

Komplexe Gruppierung

Verdichtung durch Gruppierung und Aggregation



- **`GROUP BY`**-Klausel steuert Verdichtungsgrad
  - DRILL DOWN: *Hineinnavigieren* durch feinere Granularität bzw. mehr Attribute in der **`GROUP BY`**-Klausel (= geringerer Verdichtungsgrad)
  - ROLL UP: *Herausnavigieren* durch gröbere Granularität bzw. weniger Attribute in der **`GROUP BY`**-Klausel (= höherer Verdichtungsgrad)
- Erweiterung der **`GROUP BY`**-Klausel mit Operatoren für **analytische Anfragen**
  - **`ROLLUP`**: berechnet Teil- und Gesamtsummen (SQL:1999)
  - **`CUBE`**: generiert alle möglichen Gruppierungskombinationen und berechnet Teil- und Gesamtsummen (SQL:1999)
  - **`GROUPING SETS`**: berechnet Teil- und Gesamtsummen für ausgewählte Gruppierungskombinationen (SQL:1999)
  - **`PIVOT/UNPIVOT`**: vertauschen von Zeilen und Spalten (Kein SQL-Standard – aber weit verbreitet – Oracle/SQL Server)

© 2026

Überblick

- Komplexe Gruppierung
  - `ROLLUP`- und `CUBE`-Operator
  - `GROUPING SETS`- und `GROUPING`-Operator
  - `PIVOT`- und `UNPIVOT`-Operator
- Analytische Anfragen
  - `OVER`- und `PARTITION BY`-Klausel
  - `OVER`- und `ORDER BY`-Klausel
  - Ranking-Funktionen
- Rekursive Anfragen
- Künstliche Schlüssel und Schlüsselgenerierung
  - Sequenzgeneratoren
  - Identitätsspalten
- Generierte Spalten
- Ausgewählte Funktionen und Anweisungen

© 2026

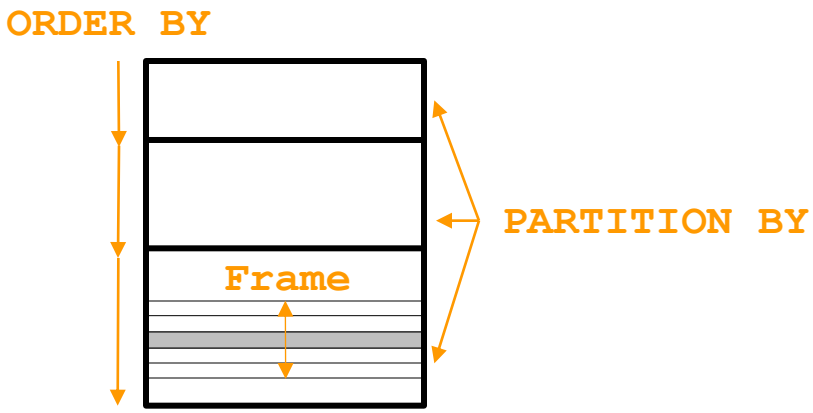
Analytische Anfragen

- **SQL:2003** führt `OVER`- und `PARTITION BY`-Klauseln ein
- Einsatz im **Data Warehouse-Bereich** für **analytische Datenauswertungen**
- Funktionen werden auf **Partitionen von Zeilen** – sogenannten **Windows** – angewendet und vereinfachen Anfragen mit
  - Zeitreihenanalyse
  - gleitenden Durchschnitten
  - kumulierten Gesamtsummen
  - Ranking (Bewertung)
  - Perzentil/Quartile-Klassifizierung (Unterteilung geordneter Datensätze)
  - etc.

© 2026

Analytische Anfragen
Window-Funktionen – Konzept

- Window-Funktionen führen eine Berechnung über eine Menge von Zeilen durch, die gemäß eines Ausdrucks (meist Attribute) in logische Gruppen (`PARTITION BY`-Klausel) zusammengefasst sind.
- Im Gegensatz zur `GROUP BY`-Klausel werden die Tupel einer Partition nicht zu einem einzigen Ergebnistupel aggregiert – die Zeilen bleiben erhalten.



Konzept der Window-Funktion:
Partitionierung, Sortierung, Framing

Analytische Anfragen

Window-Funktionen – Grundgerüst **SQL:2003**

Notation (Auszug):

```
FUNCTION_NAME(expr) OVER ( window_name | (window_spec) )

window_spec  ::= [window_name] [partitioning] [ordering] [framing]
partitioning ::= PARTITION BY value {, value}
ordering     ::= ORDER BY rule {, rule}
rule         ::= [ASC | DESC] [NULLS (FIRST | LAST)]
framing      ::= (ROWS | RANGE) (start | between) [exclusion]
between      ::= BETWEEN start AND end
start        ::= (UNBOUNDED PRECEDING | n PRECEDING | CURRENT ROW)
end          ::= (UNBOUNDED FOLLOWING | n FOLLOWING | CURRENT ROW)
exclusion    ::= EXCLUDE (CURRENT ROW | GROUP | TIES | NO OTHERS)
```

Die einfachste Form ist somit ein **FUNCTION_NAME(expr) OVER(…)** – z.B.:

```
AVG(salary) OVER(…);
```

Analytische Anfragen

Window-Funktionen – Übersicht

```
FUNCTION_NAME ( ──────────────
```

Analytische Anfragen

Ausgangstabelle

Ausgangstabelle mit Anfrage:

```
SELECT last_name, job_id, salary
FROM Employees;
```



	LAST_NAME	JOB_ID	SALARY
1	King	AD_PRES	24000
2	Kochhar	AD_VP	17000
3	De Haan	AD_VP	17000
4	Hunold	IT_PROG	9000
5	Ernst	IT_PROG	6000
6	Lorentz	IT_PROG	4200
7	Mourgos	ST_MAN	5800
8	Rajs	ST_CLERK	3500
9	Davies	ST_CLERK	3100
10	Matos	ST_CLERK	2600
11	Vargas	ST_CLERK	2500
12	Zlotkey	SA_MAN	10500
13	Abel	SA_REP	11000
14	Taylor	SA_REP	8600
15	Grant	SA_REP	7000
16	Whalen	AD_ASST	4400
17	Hartstein	MK_MAN	13000
18	Fay	MK_REP	6000
19	Higgins	AC_MGR	12000
20	Gietz	AC_ACCOUNT	8300

© 2026

Analytische Anfragen

Gruppierung mit Aggregation

Gruppierung mit Aggregation:

Erstellen Sie eine Anfrage, welche das durchschnittliche Gehalt je Berufsbezeichnung ermittelt?

```
SELECT job_id, AVG(salary)
FROM Employees
GROUP BY job_id;
```



	JOB_ID	TRUNC(AVG(SALARY),2)
1	IT_PROG	6400
2	AC_MGR	12000
3	AC_ACCOUNT	8300
4	ST_MAN	5800
5	AD_ASST	4400
6	AD_VP	17000
7	SA_MAN	10500
8	MK_MAN	13000
9	AD_PRES	24000
10	SA_REP	8866,66
11	MK_REP	6000
12	ST_CLERK	2925

Soweit nichts neues: Hier wird **`AVG()`** als Aggregatsfunktion in der Gruppierung verwendet, d.h. die einzelnen MitarbeiterInnen-Tupel gehen verloren.

Analytische Anfragen
Gruppierung mit Self Join

Gruppierung mit Self Join:

Erstellen Sie eine Anfrage, welche **alle MitarbeiterInnen** mit Nachnamen, Berufsbezeichnung, Gehalt sowie **mit dem durchschnittlichen Gehalt der jeweiligen Berufsbezeichnung** ermittelt?

```
WITH avereage_salary_by_job_id
  AS ( SELECT job_id, AVG(salary) avg_sal
       FROM Employees
       GROUP BY (job_id))

SELECT  last_name, e.job_id, salary,
        avg_sal
FROM Employees e
  INNER JOIN avereage_salary_by_job_id a
  ON (e.job_id = a.job_id);
```



	LAST_NAME	JOB_ID	SALARY	AVG_SAL
1	King	AD_PRES	24000	24000
2	Kochhar	AD_VP	17000	17000
3	De Haan	AD_VP	17000	17000
4	Hunold	IT_PROG	9000	6400
5	Ernst	IT_PROG	6000	6400
6	Lorentz	IT_PROG	4200	6400
7	Mourgos	ST_MAN	5800	5800
8	Rajs	ST_CLERK	3500	2925
9	Davies	ST_CLERK	3100	2925
10	Matos	ST_CLERK	2600	2925
11	Vargas	ST_CLERK	2500	2925
12	Zlotkey	SA_MAN	10500	10500
13	Abel	SA_REP	11000	8866,66...
14	Taylor	SA_REP	8600	8866,66...
15	Grant	SA_REP	7000	8866,66...
16	Whalen	AD_ASST	4400	4400
17	Hartstein	MK_MAN	13000	13000
18	Fay	MK_REP	6000	6000
19	Higgins	AC_MGR	12000	12000
20	Gietz	AC_ACCOUNT	8300	8300

© 2026

Analytische Anfragen

OVER- mit PARTITION BY-Klausel

Window-Funktion mit Partitionierung:

Erstellen Sie eine Anfrage, welche **alle MitarbeiterInnen** mit Nachnamen, Berufsbezeichnung, Gehalt sowie **mit dem durchschnittlichen Gehalt der jeweiligen Berufsbezeichnung** ermittelt?

```
SELECT last_name, job_id, salary,
AVG(salary) OVER(PARTITION BY job_id) AS avg_sal
FROM Employees;
```



	LAST_NAME	JOB_ID	SALARY	AVG_SAL
1	Gietz	AC_ACCOUNT	8300	8300
2	Higgins	AC_MGR	12000	12000
3	Whalen	AD_ASST	4400	4400
4	King	AD_PRES	24000	24000
5	De Haan	AD_VP	17000	17000
6	Kochhar	AD_VP	17000	17000
7	Hunold	IT_PROG	9000	6400
8	Ernst	IT_PROG	6000	6400
9	Lorentz	IT_PROG	4200	6400
10	Hartstein	MK_MAN	13000	13000
11	Fay	MK_REP	6000	6000
12	Zlotkey	SA_MAN	10500	10500
13	Abel	SA_REP	11000	8866,66...
14	Grant	SA_REP	7000	8866,66...
15	Taylor	SA_REP	8600	8866,66...
16	Matos	ST_CLERK	2600	2925
17	Davies	ST_CLERK	3100	2925
18	Rajs	ST_CLERK	3500	2925
19	Vargas	ST_CLERK	2500	2925
20	Mourgos	ST_MAN	5800	5800

© 2026

Analytische Anfragen

OVER- ohne PARTITION BY-Klausel

Window-Funktion ohne Partitionierung:

Erstellen Sie eine Anfrage, welche alle MitarbeiterInnen mit Nachnamen, Berufsbezeichnung, Gehalt sowie **mit dem durchschnittlichen Gehalt aller MitarbeiterInnen** ermittelt?

```
SELECT last_name, job_id, salary,
AVG(salary) OVER() AS avg_sal
FROM Employees;
```

	LAST_NAME	JOB_ID	SALARY	DURCHSCHNITT
1	King	AD_PRES	24000	8775
2	Kochhar	AD_VP	17000	8775
3	De Haan	AD_VP	17000	8775
4	Hunold	IT_PROG	9000	8775
5	Ernst	IT_PROG	6000	8775
6	Lorentz	IT_PROG	4200	8775
7	Mourgos	ST_MAN	5800	8775
8	Rajs	ST_CLERK	3500	8775
9	Davies	ST_CLERK	3100	8775
10	Matos	ST_CLERK	2600	8775
11	Vargas	ST_CLERK	2500	8775
12	Zlotkey	SA_MAN	10500	8775
13	Abel	SA_REP	11000	8775
14	Taylor	SA_REP	8600	8775
15	Grant	SA_REP	7000	8775
16	Whalen	AD_ASST	4400	8775
17	Hartstein	MK_MAN	13000	8775
18	Fay	MK_REP	6000	8775
19	Higgins	AC_MGR	12000	8775
20	Gietz	AC_ACCOUNT	8300	8775

Lässt man die PARTITION BY-Klausen weg, gibt es nur eine Partition (nämlich alle Zeilen). Der Durchschnitt ist somit über alle Werte.

© 2026

Analytische Anfragen
OVER-Klausel in a Nutshell

- **OVER**-Klausel kann mit allen **Aggregatfunktionen** (`MIN`, `MAX`, `AVG`, `MEDIAN`, etc.) verwendet werden

- **OVER**-Klausel definiert über welche Zeilengruppen Aggregate gebildet werden:
  - **`OVER()`**: komplette Ergebnismenge wird als eine logische Gruppe betrachtet
  - **`OVER(PARTITION BY ...)`** : gruppiert Ergebnismenge nach angegeben Kriterien, ähnlich der GROUP BY-Klausel

- **OVER**-Klausel reduziert die Tupel einer Gruppe (im Gegensatz zu **`GROUP BY`**) nicht zu einem einzigen Ergebnistupel

© 2026

Analytische Anfragen
Ausgangstabelle

Ausgangstabelle **TRANSACTION** mit Anfrage:

```
SELECT transaction_id, account_id, value
FROM Transaction;
```



	TRANSACTION_ID	ACCOUNT_ID	VALUE
1	1	100	10
2	3	100	-5
3	2	200	30
4	5	100	50
5	4	100	20
6	7	200	15
7	6	100	-20
8	8	200	-10

© 2026

Analytische Anfragen
Skalare Unterabfrage

Skalare Unterabfrage zur Ermittlung der kumulativen Gesamtsumme:

Erstellen Sie eine Anfrage, welche den laufenden Saldo nach durchgeführten Transaktionen zu einem Konto (`account_id = 100`) ermittelt?

```
SELECT transaction id, value,
 (SELECT SUM(value)
    FROM Transaction tx2
    WHERE tx2.account_id = tx1.account_id
    AND tx2.transaction_id <= tx1.transaction_id) AS balance
FROM Transaction tx1
WHERE account_id = 100
ORDER BY transaction_id;
```



	TRANSACTION_ID	VALUE	BALANCE
1	1	10	10
2	3	-5	5
3	4	20	25
4	5	50	75
5	6	-20	55

Range Segregation (<=) mit **GROUP BY** oder **PARTITION BY** nicht möglich!

Analytische Anfragen
OVER- mit ORDER BY-Klausel

OVER- mit ORDER BY-Klausel:

Erstellen Sie eine Anfrage, welche den laufenden Saldo nach durchgeführten Transaktionen zu einem Konto (`account_id = 100`) ermittelt?

```
SELECT transaction_id, value,
  SUM(value)
  OVER(
    ORDER BY transaction_id
  ) AS balance
FROM Transaction
WHERE account_id = 100;
```

	TRANSACTION_ID	VALUE	BALANCE
1	1	10	10
2	3	-5	5
3	4	20	25
4	5	50	75
5	6	-20	55

Analytische Anfragen
OVER- mit ORDER BY-Klausel

OVER- mit ORDER BY-Klausel:

Erstellen Sie eine Anfrage, welche den laufenden Saldo nach durchgeführten Transaktionen zu einem Konto (`account_id = 100`) ermittelt?

```
SELECT transaction_id, value,
  SUM(value)
  OVER(
    ORDER BY transaction_id
      ROWS BETWEEN
           UNBOUNDED PRECEDING

  ) AS balance
FROM Transaction
WHERE account_id = 100;
```

	TRANSACTION_ID	VALUE	BALANCE
1	1	10	10
2	3	-5	5
3	4	20	25
4	5	50	75
5	6	-20	55

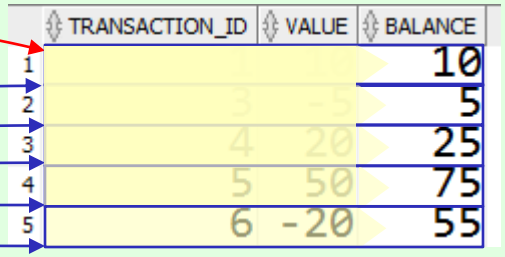
Analytische Anfragen

OVER- mit ORDER BY-Klausel

OVER- mit ORDER BY-Klausel:

Erstellen Sie eine Anfrage, welche den laufenden Saldo nach durchgeführten Transaktionen zu einem Konto (`account_id = 100`) ermittelt?

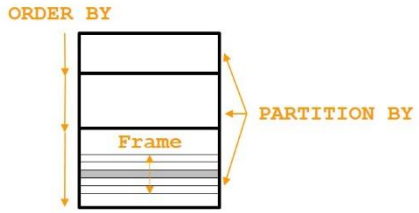
```
SELECT transaction_id, value,
  SUM(value)
  OVER(
    ORDER BY transaction_id
      ROWS  BETWEEN
            UNBOUNDED PRECEDING
        AND CURRENT ROW
  ) AS balance
FROM Transaction
WHERE account_id = 100;
```



	TRANSACTION_ID	VALUE	BALANCE
1			10
2	3	-5	5
3	4	20	25
4	5	50	75
5	6	-20	55

Analytische Anfragen

OVER- mit PARTITION BY- und ORDER BY-Klausel



OVER- mit PARTITION BY- und ORDER BY-Klausel:

Erstellen Sie eine Anfrage, welche den laufenden Saldo nach durchgeführten Transaktionen zu allen Konten ermittelt?

```
SELECT account_id, transaction_id, value,
  SUM(value)
  OVER(PARTITION BY account_id
    ORDER BY transaction_id
     ROWS  BETWEEN
           UNBOUNDED PRECEDING
      AND  CURRENT ROW

  ) AS balance
FROM Transaction;
```

Partitionierung

Sortierung

Framing

	ACCOUNT_ID	TRANSACTION_ID	VALUE	BALANCE
1	100	1	10	10
2	100	3	-5	5
3	100	4	20	25
4	100	5	50	75
5	100	6	-20	55
6	200	2	30	30
7	200	7	15	45
8	200	8	-10	35

© 2026

Analytische Anfragen

ORDER BY-Klausel

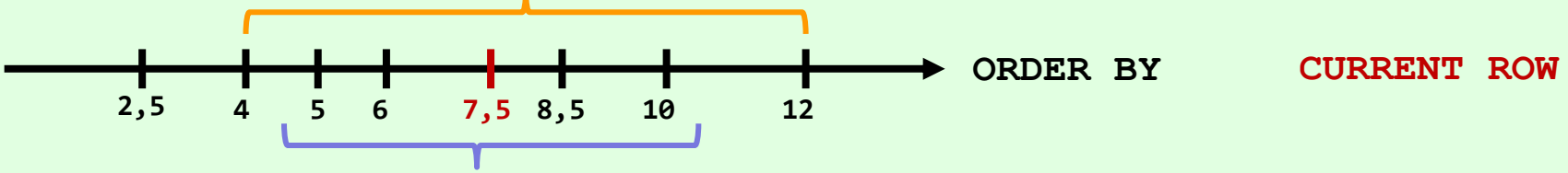
Notation (Auszug):

```
FUNCTION_NAME(expr) OVER ( window_name | (window_spec) )

window_spec  ::= [window_name] [partitioning] [ordering] [framing]
partitioning ::= PARTITION BY value {, value}
ordering     ::= ORDER BY rule {, rule}
rule         ::= [ASC | DESC] [NULLS (FIRST | LAST)]
framing      ::= (ROWS | RANGE) (start | between) [exclusion]
between      ::= BETWEEN start AND end
start        ::= (UNBOUNDED PRECEDING | n PRECEDING | CURRENT ROW)
end          ::= (UNBOUNDED FOLLOWING | n FOLLOWING | CURRENT ROW)
exclusion    ::= EXCLUDE (CURRENT ROW | GROUP | TIES | NO OTHERS)
```

ROWS und RANGE:

ROWS BETWEEN 3 PRECEDING AND 3 FOLLOWING



RANGE BETWEEN 3 PRECEDING AND 3 FOLLOWING

Analytische Funktionen
Ranking-Funktionen

Window-Funktionen: DENSE_RANK, RANK und ROW_NUMBER:

```
SELECT last_name, job_id, salary,
DENSE_RANK() OVER(PARTITION BY job_id ORDER BY salary DESC) AS d_rank,
RANK() OVER(PARTITION BY job_id ORDER BY salary DESC) AS rank,
ROW_NUMBER() OVER(PARTITION BY job_id ORDER BY salary DESC) AS row_num
FROM Employees
WHERE job_id IN ('ST_CLERK', 'AD_VP');
```



	LAST_NAME	JOB_ID	SALARY	D_RANK	RANK	ROW_NUM
1	De Haan	AD_VP	17000	1	1	1
2	Kochhar	AD_VP	15000	2	2	2
3	Rajs	ST_CLERK	3500	1	1	1
4	Davies	ST_CLERK	2600	2	2	2
5	Matos	ST_CLERK	2600	2	2	3
6	Vargas	ST_CLERK	2500	3	4	4

- **RANK()**: Rang des aktuellen Tupels mit Lücken (gleichrangige Tupel erhalten denselben Wert; nachfolgenden wird aber ein höherer Rangwert zugeordnet)
- **DENSE_RANK()**: Rang des aktuellen Tupels ohne Lücken.
- **ROW_NUMBER()**: Zeilennummer des aktuellen Tupels.

© 2026

Analytische Funktionen

Ranking-Funktionen mit FIRST und LAST

	EMPLOYEE_ID	FIRST_NAME	LAST_NAME	SALARY	DEPARTMENT_ID
1	103	Alexander	Hunold	9000	60
2	104	Bruce	Ernst	6000	60
3	107	Diana	Lorentz	4200	60
4	124	Kevin	Mourgos	6000	50
5	141	Trenna	Rajs	6000	50
6	142	Curtis	Davies	3100	50
7	143	Randall	Matos	500	50
8	144	Peter	Vargas	500	50

Ausgangstabelle **copy_employee**

Window-Funktionen: DENSE_RANK und FIRST, LAST:

```
SELECT
    department_id,
    last_name,
    first_name,
    salary,
    MIN(salary) OVER (PARTITION BY department_id) dep_min_salary,
    MAX(salary) OVER (PARTITION BY department_id) dep_max_salary,
    COUNT(*) OVER (PARTITION BY department_id) num_employees,
    COUNT(employee_id) KEEP (DENSE_RANK FIRST ORDER BY salary)
       OVER (PARTITION BY department_id) num_emp_in_dep_min_salary,
    COUNT(employee_id) KEEP (DENSE_RANK LAST ORDER BY salary)
       OVER (PARTITION BY department_id) num_emp_in_dep_max_salary
FROM copy_employee
WHERE department_id IN (50, 60);
```

	DEPARTMENT_ID	LAST_NAME	FIRST_NAME	SALARY	DEP_MIN_SALARY	DEP_MAX_SALARY	NUM_EMPLOYEES	NUM_EMP_IN_DEP_MIN_SALARY	NUM_EMP_IN_DEP_MAX_SALARY
1	50	Mourgos	Kevin	6000	500	6000	5	2	2
2	50	Matos	Randall	500	500	6000	5	2	2
3	50	Davies	Curtis	3100	500	6000	5	2	2
4	50	Vargas	Peter	500	500	6000	5	2	2
5	50	Rajs	Trenna	6000	500	6000	5	2	2
6	60	Lorentz	Diana	4200	4200	9000	3	1	1
7	60	Ernst	Bruce	6000	4200	9000	3	1	1
8	60	Hunold	Alexander	9000	4200	9000	3	1	1

Aggregatsfunktion

Ranking-Funktionen mit FIRST und LAST

	EMPLOYEE_ID	FIRST_NAME	LAST_NAME	SALARY	DEPARTMENT_ID
1	103	Alexander	Hunold	9000	60
2	104	Bruce	Ernst	6000	60
3	107	Diana	Lorentz	4200	60
4	124	Kevin	Mourgos	6000	50
5	141	Trenna	Rajs	6000	50
6	142	Curtis	Davies	3100	50
7	143	Randall	Matos	500	50
8	144	Peter	Vargas	500	50

Ausgangstabelle **copy_employee**

Window-Funktionen: DENSE_RANK und FIRST, LAST:

```
SELECT
    department_id,
    MIN(salary) min_salary,
    MAX(salary) max_salary,
    COUNT(*) num_employees,
    COUNT(employee_id) KEEP (DENSE_RANK FIRST ORDER BY salary)
                        num_emp_min_salary,
    COUNT(employee_id) KEEP (DENSE_RANK LAST ORDER BY salary)
                        num_emp_max_salary
FROM copy_employee
WHERE department_id IN (50, 60)
GROUP BY department_id;
```



	DEPARTMENT_ID	MIN_SALARY	MAX_SALARY	NUM_EMPLOYEES	NUM_EMP_MIN_SALARY	NUM_EMP_MAX_SALARY
1	50	500	6000	5	2	2
2	60	4200	9000	3	1	1



Analytische Funktionen

Navigation innerhalb der Partition

Window-Funktionen: `LAG` und `LEAD`:

```
SELECT last_name, job_id, salary,
LAG(salary, 2, NULL) OVER(ORDER BY salary DESC) AS two_before,
LEAD(last_name, 1, NULL) OVER(ORDER BY salary DESC) AS next
FROM Employees
WHERE job_id IN ('ST_CLERK', 'AD_VP');
```



	LAST_NAME	JOB_ID	SALARY	TWO_BEFORE	NEXT
1	De Haan	AD_VP	17000	NULL	Kochhar
2	Kochhar	AD_VP	15000	NULL	Rajs
3	Rajs	ST_CLERK	3500	17000	Davies
4	Davies	ST_CLERK	2600	15000	Matos
5	Matos	ST_CLERK	2600	3500	Vargas
6	Vargas	ST_CLERK	2500	2600	NULL

- **`LAG(expr, offset, default)`** evaluiert `expr` für das Tupel, welches in der Partition `offset` Positionen vor dem aktuellen Tupel liegt. Der `default`-Wert wird verwendet, wenn es (wegen Grenzüberschreitung) kein derartiges Tupel gibt.
- **`LEAD(expr, offset default)`** evaluiert `expr` für das Tupel, das in der Partition `offset` Positionen nach dem aktuellen Tupel liegt Wenn es kein solches Tupel gibt, wird der angegebene `default`-Wert für die Berechnung verwendet.

© 2026

Analytische Funktionen

Window-Funktionen (Auswahl)

Funktion	Beschreibung
ROW_NUMBER()	Zeilennummer des aktuellen Tupels
RANK()	Rang des aktuellen Tupels mit Lücken (gleichrangige Tupel erhalten denselben Wert; nachfolgenden wird aber ein höherer Rangwert zugeordnet)
DENSE_RANK()	Rang des aktuellen Tupels ohne Lücken.
PERCENT_RANK()	Relativer Rang der aktuellen Zeile: (rank-1)/(total rows-1)
CUME_DIST()	Relativer Rang der aktuellen Zeile: (number of rows preceding or peer with current row) / (total rows)
NTILE(num_buckets)	Ganzzahl von 1 bis *num_buckets*, die die Partition so gleich wie möglich aufteilt.
LAG(*value* [,*offset* [,*default*]])	Retourniert den Wert der Spalte *value*, der *offset* Zeilen weiter hinten kommt (*offset* defaults to 1; *default* ist standardmäßig NULL)
LEAD(*value* [,*offset* [,*default*]])	Retourniert den Wert der Spalte *value*, der *offset* Zeilen weiter vorne steht (*offset* defaults to 1)
FIRST_VALUE(*value*)	Gibt den ersten Wert innerhalb eines Window-Frames zurück.
LAST_VALUE(*value*)	Gibt letzten Wert innerhalb eines Window-Frames zurück.

© 2026

Analytische Funktionen
Zusammenfassung

- **OVER**-Klausel ermöglicht Gruppierungen mit Aggregaten ohne **GROUP BY**
- **OVER**-Klausel **vermeidet Self Joins**

- Anwendungsbeispiele:
  - Kumulierte Gesamtsumme
  - Gleitender Durchschnitt

```
AVG(…)  OVER(ORDER BY …
             ROWS BETWEEN 3 PRECEDING
             AND 3 FOLLOWING
        ) AS moving_avg
```

  - Ranking
    - Top-N pro Gruppe

```
SELECT *
FROM ( SELECT ROW_NUMBER()
          OVER( PARTITION BY …
                ORDER BY …) AS rn,
          t.*
       FROM t)
WHERE rn <= 3
```

© 2026

Review-Fragen

- `OVER`?
- Partitionierung?
- Ordering?
- Framing/Windowing?
- Funktionen?

© 2026

Überblick

- Komplexe Gruppierung
  - `ROLLUP`- und `CUBE`-Operator
  - `GROUPING SETS`- und `GROUPING`-Operator
  - `PIVOT`- und `UNPIVOT`-Operator
- Analytische Anfragen
  - `OVER`- und `PARTITION BY`-Klausel
  - `OVER`- und `ORDER BY`-Klausel
  - Ranking-Funktionen
- Rekursive Anfragen
- Künstliche Schlüssel und Schlüsselgenerierung
  - Sequenzgeneratoren
  - Identitätsspalten
- Generierte Spalten
- Ausgewählte Funktionen und Anweisungen

© 2026

Rekursive Anfragen

Motivation – Abfragen hierarchischer Beziehungen

- Stücklistenanfragen (Teile-von-Beziehung)

Maschine

Baugruppe	Bauteil	Anzahl	Teilkosten
Auto	Motor	1	3453
Motor	Kolben	4	124
Motor	Kurbelwelle	1	67
Motor	Getriebe	1	654
Getriebe	Kupplung	1	290
...	...	...	...

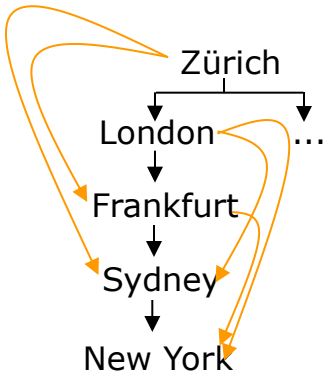
Welche mittelbaren und unmittelbaren Unterteile gehören zu welchen Oberteilen?

- Wegesuche (Berechnung transitive Hülle)

Linienflug

Flugnr	Abflug	Ziel	Preis
LH131	Frankfurt	Sydney	1009
BA170	Zürich	London	599
AF691	Paris	New York	1299
UA103	Sydney	New York	1549
SR163	Zürich	Frankfurt	499
TK911	Paris	Istanbul	799
BA297	London	Frankfurt	299

Transitive Hülle = Direkte und indirekte Flugverbindungen



Rekursive Anfragen

Beispiel

```
SELECT Abflug, Ziel              -- Direktflüge
FROM Linienflug
WHERE Abflug = 'Zürich'
UNION ALL                        -- 2-Stopp-Flüge
SELECT F1.Abflug, F2.Ziel
FROM Linienflug F1, Linienflug F2
WHERE       F1.Abflug = 'Zürich'
                AND F1.Ziel = F2.Abflug
UNION ALL                        -- 3-Stopp-Flüge
SELECT F1.Abflug, F3.Ziel
FROM Linienflug F1, Linienflug F2, Linienflug F3
WHERE       F1.Abflug = 'Zürich'
                AND F1.Ziel = F2.Abflug
                AND F2.Ziel = F3.Abflug
UNION ALL                        -- 4-Stopp-Flüge
SELECT F1.Abflug, F4.Ziel
FROM Linienflug F1, Linienflug F2, Linienflug F3, Linienflug F4
WHERE       F1.Abflug = 'Zürich'
                AND F1.Ziel = F2.Abflug
                AND F2.Ziel = F3.Abflug
            AND F3.Ziel = F4.Abflug;
```

Linienflug

Flugnr	Abflug	Ziel	Preis
LH131	Frankfurt	Sydney	1009
BA170	Zürich	London	599
AF691	Paris	New York	1299
UA103	Sydney	New York	1549
SR163	Zürich	Frankfurt	499
TK911	Paris	Istanbul	799
BA297	London	Frankfurt	299

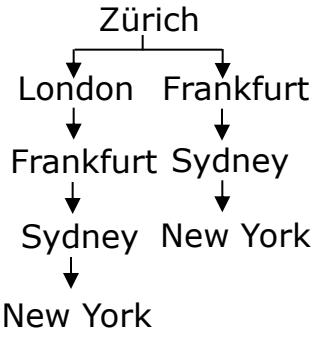
- Ermittle alle von Zürich aus direkt erreichbaren Ziele!

```
SELECT Abflug, Ziel
FROM Linienflug
WHERE Abflug = 'Zürich';
```

Abflug	Ziel
Zürich	London
Zürich	Frankfurt

- Ermittle alle von Zürich aus indirekt erreichbaren Ziele!

```
SELECT Abflug, Ziel
FROM Linienflug
WHERE Abflug = IN ( SELECT Ziel
                    FROM Linienflug
                    WHERE Abflug = 'Zürich');
```



Abflug	Ziel
London	Frankfurt
Frankfurt	Sydney

- Ermittlung aller von Zürich aus direkt oder indirekt erreichbaren Ziele kann bei einer beliebigen Anzahl von Umsteigevorgängen in SQL nicht formuliert werden - d.h. die Anzahl der Verbunde steht im Voraus nicht fest.

REPORT

Abflug	Ziel
Zürich	London
Zürich	Frankfurt
Zürich	Sydney
Zürich	New York

© 2026

Rekursive Anfragen
Grundgerüst (**SQL:1999**)

- Formulierung über erweiterte **WITH RECURSIVE**-Anfrage

```
WITH RECURSIVE rekursionsrelation (spaltennamenliste) AS
( anfrage-ausdruck )                          → rekursiver Teil
[traversierungsklausel] [zyklusklausel]
anfrage-ausdruck;                             → nicht rekursiver Teil
```

```
-- Rekursionsinitialisierung
SELECT ...
FROM basisrelation
[WHERE initialisierungsprädikat]
-- Rekursionsschritt
UNION ALL
SELECT ...
FROM basisrelation, rekursionsrelation
[WHERE rekursionsbedingung]
```

Rekursive Anfragen
Beispiel

```
WITH RECURSIVE Flugstrecke(Abflug, Ziel) AS
(
  SELECT Abflug, Ziel
  FROM Linienflug
  WHERE Abflug = 'Zürich'
  UNION ALL
  SELECT e.Abflug, f.Ziel
  FROM Linienflug e, Flugstrecke f
  WHERE e.Ziel = f.Abflug
)
SELECT DISTINCT *
FROM Flugstrecke;
```

① Rekursionsinitialisierung

Abflug	Ziel
Zürich	London
Zürich	Frankfurt

② Rekursionsschritt 1

Abflug	Ziel
Zürich	London
Zürich	Frankfurt
London	Frankfurt
Frankfurt	Sydney

② Rekursionsschritt 2

Abflug	Ziel
Zürich	London
Zürich	Frankfurt
London	Frankfurt
Frankfurt	Sydney
Frankfurt	Sydney
Sydney	New York

...

① Rekursionsinitialisierung berechnet alle direkten Verbindungen in der Rekursionsrelation `Flugstrecke`

② Rekursionsschritt berechnet alle indirekten Verbindungen basierend auf den bereits berechneten Zielorten. Die Rekursion endet, wenn in einem Rekursionsschritt keine weiteren Tupel mehr hinzukommen.

Rekursive Anfragen
Implementierungen

- Standardisierung erst mit SQL:1999
- Implementierungen in konkreten Systemen schon vor der Standardisierung vorhanden!
- Inkompatibilitäten zwischen Systemen
  - IBM DB2, MS SQL Server verwenden kein Schlüsselwort **`RECURSIVE`**
  - Oracle besitzt eigene Befehle für hierarchische Anfragen und unterstützt rekursive Anfragen auf azyklische Strukturen.

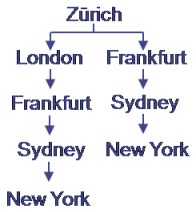
Rekursive Anfragen

Implementierung am Beispiel von Oracle

- Rekursive Anfragen auf **azyklischen Strukturen**

```
SELECT Abflug, Ziel
FROM Linienflug
START WITH Abflug = 'Zürich'
CONNECT BY PRIOR Ziel = Abflug;
```

ABFLUG	ZIEL
Zürich	Frankfurt
Frankfurt	Sydney
Sydney	New York
Zürich	London
London	Frankfurt
Frankfurt	Sydney
Sydney	New York



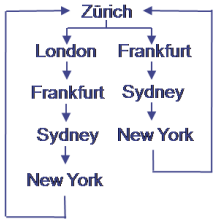
Rekursionsbedingung: gibt die Spalten an, in denen die `PRIOR`-Beziehung zwischen Vater- und Kindknoten besteht (tritt ein Zyklus in der hierarchischen Anfrage auf, bricht Oracle die Berechnung mit einer Fehlermeldung ab)

Rekursionsinitialisierung (optional): gibt die Anfangsknoten der Hierarchie an (bei denen begonnen werden soll)

- Rekursive Anfragen auf **zyklischen Strukturen**

```
SELECT Abflug, Ziel, CONNECT_BY_ISCYCLE
FROM Linienflug
START WITH Abflug = 'Zürich'
CONNECT BY NOCYCLE PRIOR Ziel = Abflug;
```

ABFLUG	ZIEL	CONNECT_BY_ISCYCLE
Zürich	Frankfurt	0
Frankfurt	Sydney	0
Sydney	New York	0
New York	Zürich	1
Zürich	London	1
Zürich	London	0
London	Frankfurt	0
Frankfurt	Sydney	0
Sydney	New York	0
New York	Zürich	1

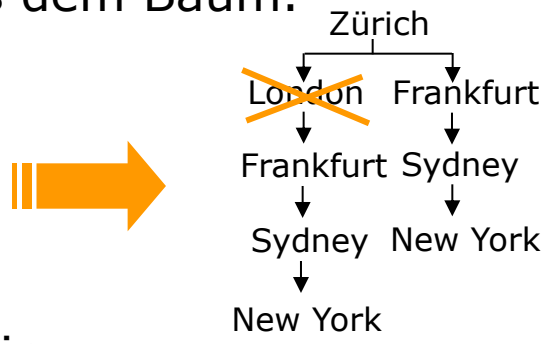


Rekursive Anfragen

Implementierung am Beispiel von Oracle

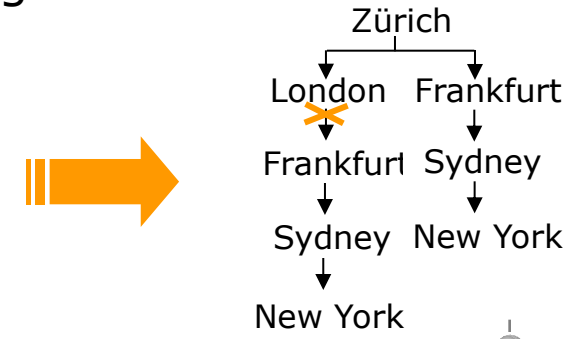
- **PRIOR** erzwingt Auflistung vom Stamm in Richtung Blätter (wenn PRIOR-Spalte das Elternteil ist) oder von den Blättern Richtung Stamm (wenn die PRIOR-Spalte das Kind ist).
- **WHERE-Klausel** eliminiert zwar einzelne Elemente, aber nicht deren Abkömmlinge (oder Vorfahren) aus dem Baum.

```
SELECT Abflug, Ziel
FROM Linienflug
WHERE Abflug <> 'London'
START WITH Abflug = 'Zürich'
CONNECT BY PRIOR Ziel = Abflug;
```



- Qualifikation in CONNECT BY eliminiert sowohl ein einzelnes Element als auch dessen Abkömmlinge (oder Vorfahren).

```
SELECT Abflug, Ziel
FROM Linienflug
START WITH Abflug = 'Zürich'
CONNECT BY PRIOR Ziel = Abflug
AND Abflug <> 'London';
```



© 2026

Rekursive Anfragen

Grundregeln am Beispiel Oracle

Notation:

```
SELECT ...
  FROM ...
[ WHERE Bedingungen ]
[ START WITH Bedingung ]
CONNECT BY   PRIOR attribut1  =  attribut2;
```

- **PRIOR:** erzwingt die Auflistung vom Stamm in Richtung Blätter (wenn die **PRIOR**-Spalte das Elternteil ist) oder von den Blättern in Richtung Stamm (wenn die **PRIOR**-Spalte das Kind ist).
- **WHERE:** eliminiert zwar einzelne Elemente, aber nicht deren Abkömmlinge (oder Vorfahren, wenn **PRIOR** auf der rechten Seite des Gleichheitszeichens steht) aus dem Baum.
- **CONNECT BY:** verknüpft die Eltern- und Kindknoten und sorgt für die Rekursion (tritt ein Zyklus in der hierarchischen Anfrage auf, bricht Oracle die Berechnung mit einer Fehlermeldung ab).
- **START WITH:** gibt die Anfangsknoten der Hierarchie an (Wurzelknoten).
- Oracle unterstützt eine Reihe von Pseudospalten, die die hierarchische Verarbeitung unterstützen (bspw. gibt **LEVEL** die Hierarchiestufe an)

© 2026

Review-Fragen

- Einsatzgebiete - Warum überhaupt?
- Unterschied SQL:1999 vs. Oracle
- **`CONNECTY BY PRIOR`**?
- **`NOCYCLE`**?

Überblick

- Komplexe Gruppierung
  - `ROLLUP`- und `CUBE`-Operator
  - `GROUPING SETS`- und `GROUPING`-Operator
  - `PIVOT`- und `UNPIVOT`-Operator
- Analytische Anfragen
  - `OVER`- und `PARTITION BY`-Klausel
  - `OVER`- und `ORDER BY`-Klausel
  - Ranking-Funktionen
- Rekursive Anfragen
- Künstliche Schlüssel und Schlüsselgenerierung
  - Sequenzgeneratoren
  - Identitätsspalten
- Generierte Spalten
- Ausgewählte Funktionen und Anweisungen

© 2026

Künstliche Schlüssel und Schlüsselgenerierung

- **Automatische Generierung** von **eindeutigen numerischen Werten** (seit SQL:2003)
  - z.B. zur Vergabe von künstlichen Schlüsselwerten
  - Parametrisierung: Startwert, Inkrement, Überlaufbehandlung
  - z.B. Oracle: `NEXT VALUE FOR sequenceID`
  - z.B. PostgreSQL: `nextval(sequenceID)`
- **Alternativen**
  - Typ-Zusatz
    - z.B. MySQL: `AUTO_INCREMENT`
    - z.B. Oracle: `GENERATED ALWAYS AS IDENTITY`
  - Pseudo-Typen
    - z.B. PostgreSQL: `SMALLSERIAL`, `SERIAL`

© 2026

Künstliche Schlüssel und Schlüsselgenerierung
Sequenzgenerator

- **Automatische Generierung** von **eindeutigen numerischen Werten** (seit SQL:2003)
  - z.B. zur Vergabe von künstlichen Schlüsselwerten
  - Parametrisierung: Startwert, Inkrement, Überlaufbehandlung

Sequenzgenerator (lt. Standard):

```
CREATE SEQUENCE PersonIDGen AS INTEGER
    START WITH 1000 INCREMENT BY 1
    MINVALUE 1000 MAXVALUE 99999
    CYCLE;
```

- **CYCLE**-Klausel bestimmt über den nächsten Schritt, wenn beim Inkrementieren der Maximalwert (bzw. beim Dekrementieren der Minimalwert) erreicht wird
  - **NO CYCLE** beendet Generierung mit einer Ausnahmebedingung
  - **CYCLE** setzt die Generierung mit dem Minimalwert fort

Anwendung:

```
INSERT INTO MitarbeiterTupeltabelle (MNr, Name)
  VALUES (NEXT VALUE FOR PersonIDGen, 'Harry');
```

© 2026

Künstliche Schlüssel und Schlüsselgenerierung
Identitätsspalten

- Automatische Generierung von Schlüsseln mit Hilfe eines **impliziten Sequenzgenerators** (seit SQL:2003)
- Eine Tabelle darf **maximal eine Identitätsspalte** enthalten

Identitätsspalte (Oracle):

```
CREATE TABLE Produkt (
  ProdId INTEGER GENERATED ALWAYS AS IDENTITY
  ( START WITH 200 INCREMENT BY 1
    MINVALUE 200 MAXVALUE 100000
    NOCYCLE),
  Bezeichnung VARCHAR(20), Preis DECIMAL(9,2), ...
);
```

Anwendung:

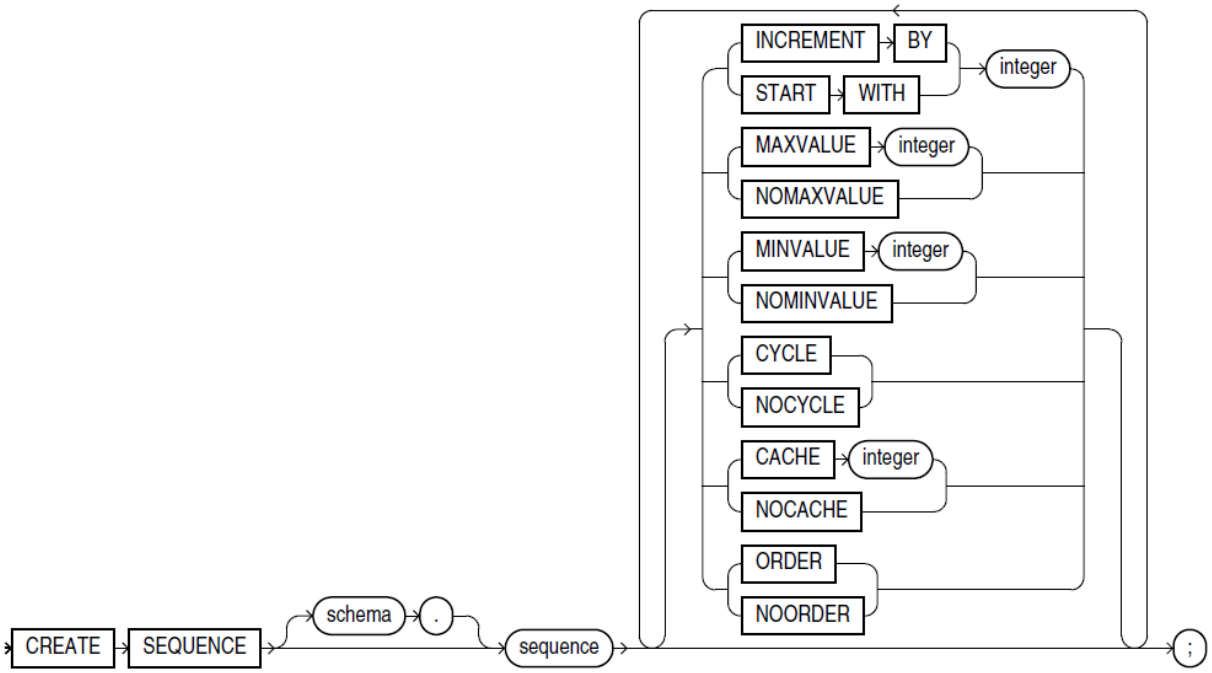
```
INSERT INTO Produkt (Bezeichnung, Preis, ...)
VALUES ( 'Messer', 20,...)
```

- Wert der generierten Spalte wird automatisch berechnet
  - z.B. DB2, MySQL, SQLServer realisieren Identitätsspalte

Künstliche Schlüssel und Schlüsselgenerierung

Implementierung am Beispiel von Oracle

- Sequenzgenerator stellt zusätzliche Konfigurations-möglichkeiten zu Verfügung (bspw. Order)
- Identitätspalten erst ab Oracle 12c



Review-Fragen

- Wozu künstliche Schlüssel bzw. warum nicht immer künstliche Schlüssel?
- Möglichkeiten der Implementierung?

© 2026

Überblick

- Komplexe Gruppierung
  - `ROLLUP`- und `CUBE`-Operator
  - `GROUPING SETS`- und `GROUPING`-Operator
  - `PIVOT`- und `UNPIVOT`-Operator
- Analytische Anfragen
  - `OVER`- und `PARTITION BY`-Klausel
  - `OVER`- und `ORDER BY`-Klausel
  - Ranking-Funktionen
- Rekursive Anfragen
- Künstliche Schlüssel und Schlüsselgenerierung
  - Sequenzgeneratoren
  - Identitätsspalten
- Generierte Spalten
- Ausgewählte Funktionen und Anweisungen

© 2026

Generierte/berechnete Spalten

- SQL:2003 unterstützt generierte Spalten mit berechneten Werten
- Attribute von generierten Spalten dürfen
  - sich nur auf Attribute der gleichen Tabelle beziehen,
  - sich nicht auf generierte Spalten beziehen,
  - nicht in einer **`INSERT`**-, **`UPDATE`**- oder **`MERGE`**-Anweisung mit Werten gesetzt werden.
- Generierte Spalten werden zur Laufzeit berechnet:
  - zum Zeitpunkt der Datenänderung (**`INSERT`**, **`UPDATE`**, **`MERGE`**)
  - oder zum Zeitpunkt der Datenabfrage (**`SELECT`**)
- Generierte Spalten können
  - physisch in der Tabelle abgelegt werden (wie bei Basisspalten)
  - oder auch nicht (**`VIRTUAL`**) (Wert wird bei Abfrage ermittelt)

© 2026

Generierte/berechnete Spalten

Generierte Spalten:

```
CREATE TABLE Artikel (
  ArtikelNr NUMBER(10)PRIMARY KEY,
  ArtikelName VARCHAR(50) NOT NULL,
  Nettopreis DECIMAL(10,2) NOT NULL,
  Ust_Satz DECIMAL(4,2) NOT NULL,
  Ust_Betrag DECIMAL(10,2) GENERATED ALWAYS AS
      (Nettopreis * Ust_Satz / 100.0) VIRTUAL,
  Bruttopreis DECIMAL(10,2) GENERATED ALWAYS AS
      (Nettopreis * (1 + Ust_Satz/100)) VIRTUAL);
```

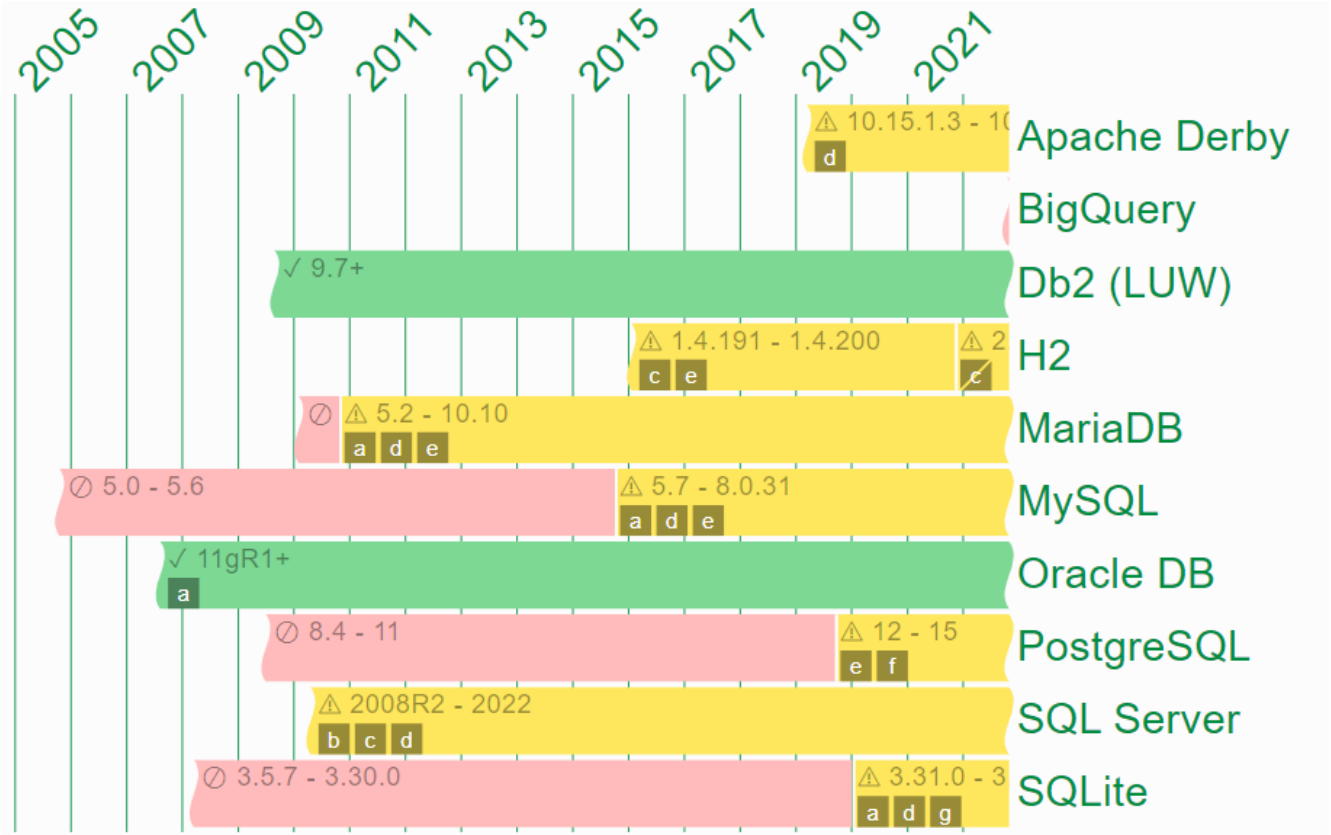
Ausdruck darf sich nicht auf andere generierten Spalten beziehen

kein `SELECT-FROM-WHERE` erlaubt!

- Vorteile gegenüber Sichten
  - auf generierten Spalten lassen sich Indizes und Constraints definieren

```
ALTER TABLE Artikel ADD CONSTRAINT Check_Bruttopreis
CHECK(Bruttopreis < 100000);
```

Generierte/berechnete Spalten



a. Unterstützt Speicher-Optionen
b. Nur ohne expliziten Datentyp: col_name integer generated always as …
c. Nur ohne die Schlüsselwörter generated always
d. Eingeschränkte Unterstützung von Constraints
e. Erfordert einen expliziten Datentyp: col_name integer generated always as …
f. Erfordert die Speicher-Option stored
g. Ignoriert den Datentyp, falls angegeben

modern SQL: https://modern-sql.com/de/gehtdas/generated-always-as#storage

Überblick

- Komplexe Gruppierung
  - `ROLLUP`- und `CUBE`-Operator
  - `GROUPING SETS`- und `GROUPING`-Operator
  - `PIVOT`- und `UNPIVOT`-Operator
- Analytische Anfragen
  - `OVER`- und `PARTITION BY`-Klausel
  - `OVER`- und `ORDER BY`-Klausel
  - Ranking-Funktionen
- Rekursive Anfragen
- Künstliche Schlüssel und Schlüsselgenerierung
  - Sequenzgeneratoren
  - Identitätsspalten
- Generierte Spalten
- Ausgewählte Anweisungen und Funktionen

© 2026

Ausgewählte Anweisungen
TRUNCATE

- seit SQL:2008
- Löschen aller Tupel aus einer Tabelle und Freigeben des belegten Speicherplatzes
  - Ähnlich zu `DROP TABLE`, aber Tabellendefinition bleibt erhalten
  - Ähnlich zu `DELETE`, kann aber nicht mit `ROLLBACK` zurückgenommen werden
  - `TRUNCATE` ist effizient implementiert und daher schnell und führt explizit ein `COMMIT` aus.!

TRUNCATE:

```
TRUNCATE TABLE Artikel;
```

- ACHTUNG:
  - es erfolgt ein explizites `COMMIT`
  - es werden keine Trigger ausgeführt

© 2026

Ausgewählte Anweisungen
FETCH FIRST

- seit SQL:2008
- unterstützt Top-N-Anfragen

Notation:

```
SELECT ...
FROM ...
[OFFSET offset {ROW|ROWS}]
[FETCH {FIRST|NEXT} [{rowcount|percent PERCENT}]
   {ROW|ROWS} {ONLY|WITH TIES}];
```

Top-N-Anfragen

```
SELECT last_name, salary
FROM Employees
ORDER BY last_name
FETCH FIRST 5 ROWS ONLY;
```

Top-N-Anfragen

```
SELECT last_name, salary
FROM Employees
ORDER BY last_name
OFFSET 5 ROWS
FETCH NEXT 5 ROWS ONLY;
```

© 2026

Ausgewählte Funktionen
LISTAGG

- seit SQL:2016
- fügt Spaltenwerte aus einer Gruppe von Zeilen zu einer Zeichenkette mit konfigurierbarem Separator zusammen

Notation:

```
SELECT LISTAGG(Ausdruck, Separator)
               WITHIN GROUP(ORDER BY ...)
               [FILTER(WHERE ...)]
               [OVER(...)]
FROM ...;
```

- `ausdruck` darf keine Aggregatsfunktionen und Unterabfragen enthalten
- `FILTER`-Klausel entfernt Zeilen vor der Aggregation
- `OVER`-Klausel legt **analytische Funktion** fest

Ausgewählte Funktionen
LISTAGG

- Gruppierung und Konkatenation aller Werte

Aggregation/Konkatenation:

```
SELECT Kunde, SUM(Menge) AS Gesamtmenge,
       LISTAGG(menge || '*' || produkt, ', ')
              WITHIN GROUP (ORDER BY produkt) AS Details
  FROM Verkauf
GROUP BY Kunde;
```

Bestimmt die Werte, die konkateniert werden

Frei wählbares Trennzeichen

Bestimmt die Sortierung der konkatenierten Werte

Verkauf

ID	Kunde	Produkt	Menge
1	Kunde A	Kalender	100
2	Kunde B	Block	40
3	Kunde C	Heft	70
4	Kunde D	Block	100
5	Kunde A	Block	200
6	Kunde D	Heft	30

Kunde	Gesamtmenge	Details
Kunde A	300	200*Block, 100*Kalender
Kunde B	40	40*Block
Kunde C	70	70*Heft
Kunde D	130	100*Block, 30*Heft

© 2026

Ausgewählte Funktionen
NVL, COALESCE

- NVL: ersetzt NULL Werte (Oracle-spezifisch)

Rechnen mit NULL

```
SELECT NVL(NULL, 0) FROM DUAL;  --> 0
SELECT NVL(3, 0) FROM DUAL;     --> 3
```

  - evaluiert immer alle Parameter

- COALESCE nimmt ersten nicht NULL Wert (SQL Standard)

Rechnen mit NULL

```
SELECT COALESCE(1, 2, 3) FROM DUAL;                        --> 1
SELECT COALESCE(NULL, NULL, 1, 2, 3, NULL) FROM DUAL;      --> 1
```

  - Kurzschlussevaluierung aller Parameter

© 2026

Zusammenfassung

- **SQL** ist die in der Praxis am **weitesten verbreitete** Datenbanksprache für relationale Datenbanken.
- **SQL-Kern** mit Bezug zur **Relationenalgebra**
- Der Sprachumfang von SQL ist einer **permanenten Weiterentwicklung** und Standardisierung unterworfen.
- **SQL:2023** repräsentiert den aktuellen Standard.
- **Datenbanksysteme** realisieren **weitgehend** die im Standard **SQL2** definierten Konzepte und bereits viele Konzepte der Standards SQL:1999, SQL:2003, SQL:2011 und SQL:2016.
- Viele Hersteller haben in ihren Systemen **über den Standard hinausgehende SQL-Erweiterungen** implementiert.

© 2026